

TÀI LIỆU KHÓA HỌC

TypeScript Pro - Tự Học TypeScript & JavaScript từ số 0

Tác giả: Hỏi Dân IT & Eric

Version: 1.0

Demo kết quả đạt được khi kết thúc khóa học

1.Mục tiêu của khóa học

Xây dựng niềm đam mê lập trình với ngôn ngữ JavaScript/TypeScript từ số 0 dành cho người mới bắt đầu.

Bạn sẽ học được gì?

- Kiến thức căn bản về lập trình: biến, kiểu dữ liệu, hàm, vòng lặp, điều kiện, object, array...
- Hiểu và thao tác với DOM, xử lý sự kiện, tương tác với giao diện HTML.
- Cách làm việc với bất đồng bộ: callback, Promise, async/await.
- Làm quen và nắm vững TypeScript: kiểu dữ liệu tĩnh, interface, class, function typing...
- Xây dựng dự án thực hành giúp bạn áp dụng ngay kiến thức đã học.

2. Demo dự án

```
//todo
```

3.Khóa học phù hợp với ai?

- Người mới học lập trình từ con số 0, muốn xây dựng đam mê lập trình
- Sinh viên, người trái ngành muốn chuyển sang IT, học với ngôn ngữ JavaScript/TypeScript
- Developer JavaScript muốn nâng cấp lên TypeScript một cách bài bản.

#Tổng quan về các kiến thức sẽ học

Khóa học bao gồm **16 chapters**, chia thành 2 mảng kiến thức chính:

Từ chapter 1 tới hết chapter 8: Học & thực hành **ngôn ngữ JavaScript** từ số 0.

Mục tiêu của phần kiến thức này, giúp bạn nắm vững kiến thức cơ bản & cốt lõi nhất của JavaScript, cũng như hiểu rõ “nhược điểm” của JavaScript (khi cần scale, maintain...)

Từ chapter 9 tới hết chapter 16: Học & thực hành **ngôn ngữ Typescript** từ số 0.

Nhờ việc nắm vững JavaScript, bạn tiếp cận TypeScript một cách dễ dàng hơn (không cần học lại cú pháp, datatype, function...) vì:

TypeScript = JavaScript + Type

Trọng tâm của khóa học là TypeScript, tuy nhiên, cần học JavaScript để thấy được lịch sử phát triển của công nghệ, nhược điểm của Javascript và lý do TypeScript ra đời.

Ngoài ra, việc nắm vững cả JavaScript/TypeScript sẽ giúp bạn thích nghi tốt nhất với cuộc sống khi đi làm, cho dù rơi vào dự án JavaScript hay TypeScript, đều có thể tham gia.

Hướng Dẫn Sử Dụng Udemy Dành Cho Người Mới Bắt Đầu

Tài liệu hướng dẫn chi tiết xem [tại đây](#)

Mục tiêu: hướng dẫn sử dụng các tính năng chủ yếu của Udemy, hỗ trợ việc học tập hiệu quả hơn, dành cho người lần đầu tiên sử dụng nền tảng Udemy.

1. Các tính năng cơ bản

- Hiển thị giao diện ngôn ngữ tiếng anh/tiếng việt
- Khóa học của tôi nằm ở đâu ? (my learning - kiểm tra các tabs)
- Trình phát video
- Điều chỉnh chất lượng, tốc độ video
- Download tài liệu (nếu có)

2. Các tính năng khác

- Bạn có thể sử dụng Udemy App trên [mobile](#). Download [tại đây](#)
- Q/A, Notes, Announcements

Cách Hỗ Trợ/Support Trong Quá Trình Học (Bắt Buộc Xem)

Mục đích:

- **Giải đáp thắc mắc** của học viên liên quan về nội dung khóa học
- **Hỗ trợ gỡ lỗi** (fix bug) trong quá trình thực hành theo video hướng dẫn

1. Tính năng Q&A (Hỏi/Đáp)

Sử dụng tính năng này khi bạn cần giải đáp thắc mắc & hỗ trợ trong quá trình thực hành, với các ưu điểm vượt trội sau:

- **Bảo vệ sự riêng tư của bạn** (chỉ hiển thị tên tài khoản Udemy)
- **Hỗ trợ upload hình ảnh, gõ code...** giao diện trực quan, dễ theo dõi, tránh tình trạng trôi tin nhắn (so với cách chat text)
- **Hỗ trợ lọc/filter câu hỏi, xem lịch sử** các câu hỏi đã đặt ứng với video đang xem
- **Cung cấp ngữ cảnh** (gặp vấn đề tại video nào), như vậy nó sẽ rõ ràng & giải quyết nhanh hơn

2. Hướng dẫn cách đặt câu hỏi

Trước khi đặt câu hỏi, bạn nên lọc câu hỏi theo video, mục đích là **kiểm tra vấn đề bạn hỏi, đã có người khác hỏi hay chưa**, giúp bạn tiết kiệm thời gian hơn.

Các nguyên tắc khi đặt câu hỏi:

- Đặt câu hỏi theo video bạn thắc mắc/cần hỗ trợ
- Câu hỏi cần rõ ràng, tường minh, tránh nêu vấn đề chung chung, không cụ thể.

Minh họa cách đặt câu hỏi:

Bước 1: Chọn bài giảng bạn “gặp vấn đề” cần hỗ trợ

Bước 2: Duyệt qua danh sách câu hỏi ứng với video đang xem (nếu có), vì nếu như đã có vấn đề bạn gặp phải, sẽ giúp bạn tiết kiệm thời gian.

Bước 3: Đặt câu hỏi mới

- **Đặt tiêu đề:** nếu ngắn gọn (đặt tên) cho vấn đề bạn muốn hỏi
- **Viết nội dung** chi tiết: miêu tả cụ thể vấn đề gặp phải

Hướng dẫn: chèn **link** (ví dụ google drive), chèn **ảnh** và gõ **code**.

Sửa (update) câu hỏi nếu cần thiết.

Gợi ý các mẫu đặt câu hỏi:

Mẫu 1 – Khi gặp lỗi code:

Tôi đang học tới **[đoạn nào trong video]**, gặp vấn đề **[Mô tả ngắn về lỗi]**.

Tôi đã thử **[Các bước đã làm để xử lý]**, nhưng kết quả là **[Thông báo lỗi hoặc tình trạng xảy ra]**.

Đây là đoạn code/hình ảnh minh họa:

[Đoạn code ngắn hoặc ảnh chụp màn hình]

Mẫu 2 - Hỏi kỹ năng:

Trong bài **[Tên bài hoặc chủ đề]**, tôi thấy phương pháp/kỹ thuật **[Tên phương pháp]** rất hay, nhưng chưa rõ cách áp dụng trong **[Ngữ cảnh thực tế của bản thân]**.

Tôi mong giảng viên/anh chị có thể cho thêm một ví dụ minh họa cụ thể để tôi dễ hình dung hơn.

3. Hướng dẫn cách xem câu trả lời

Udemy sẽ hiển thị thông báo trong tài khoản Udemy, và gửi email khi có phản hồi.

Thông thường, thời gian phản hồi câu hỏi là 24h (ngày lễ, thứ 7, chủ nhật có thể lâu hơn).
Tối đa thời gian nhận phản hồi là 02 ngày làm việc (2 business days)

Khi mở email, cần kiểm tra từ inbox (primary), update, social, spam...

//todo: hướng dẫn kéo thả email vào primary (**no-reply@e.udemymail.com**)

(Mỗi lần enroll, sẽ có email tới từ: **hello@alerts.udemy.com**)

4. Hướng dẫn cách nhận thông báo (nếu có)

Cấu hình [tại đây](#)

5. Kênh liên hệ giảng viên trên Udemy (message)

Profile giảng viên: <https://www.udemy.com/user/eric-7039/>

Hướng Dẫn Sử Dụng Udemy Dành Cho Người Mới Bắt Đầu	3
Cách Hỗ Trợ/Support Trong Quá Trình Học (Bắt Buộc Xem)	4
Chapter 1: Bắt buộc xem - Không bỏ qua chương học này	12
#1. Hướng Dẫn Sử Dụng Khóa Học Hiệu Quả	12
#2 + #3 + #4. Hướng Dẫn Truy Cập Tài Liệu Khóa Học	14
#5. Về Quyền Tác Giả	15
#7 + #8. Thông Tin Tác Giả Hỏi Dân IT	16
Chapter 2: Giới thiệu và thiết lập môi trường thực hành	17
#8. Tổng quan về chapter 2	17
#9. JavaScript là gì	17
#10. Chuyện cài đặt công cụ (Bắt Buộc Xem)	19
#11. Cài đặt & cấu hình Google Chrome	21
#12. Cài đặt môi trường code JavaScript	22
#13. Hello World với JavaScript	24
#14. Sử dụng Git để quản lý mã nguồn	25
Chapter 3: Biến, Kiểu dữ liệu và Toán tử	26
#15. Tổng quan về chapter 3	26
#16. Google Chrome Devtool (Extra)	26
#17. Biến trong JavaScript (var, let, const)	28
#18. Kiểu dữ liệu nguyên thủy (Primitive Data Types)	29
#19. Kiểu dữ liệu tham chiếu (Object Data Types)	30
#20. Toán tử trong JavaScript	31
#21. Câu lệnh console trong JavaScript	33
#22. Template String (Extra)	34
#23. Bài Tập Lab 01	35
#24. Chữa Bài Tập Lab 01	35
Chapter 4: Cấu Trúc Điều Khiển & Hàm (Function)	36
#25. Tổng quan về chapter 4	36
#26. Câu lệnh điều kiện – if / else / else if	36
#27. Câu lệnh switch / case	37
#28. Vòng lặp (for, while, do...while)	38
#29. Câu lệnh điều khiển luồng – break, continue	40
#30. Hàm (Function) trong JavaScript	41
#31. Arrow function (Extra)	42
#32. Keyword Return	42
#33. Phạm vi biến (Scope)	43
#34. Bài Tập Lab 02	45
#35. Chữa Bài Tập Lab 02	45
Chapter 5: Mảng (Array) và Đối Tượng (Object)	46
#36. Tổng quan về chapter 5	46
#37. Giới thiệu về Mảng (Array)	46
#38. Truy cập, chỉnh sửa, thêm và xóa phần tử của mảng	47
#39. Duyệt mảng với vòng lặp và forEach	48

#40. Biến đổi mảng với map()	49
#41. Lọc phần tử mảng với filter()	50
#42. Giới thiệu về Đối tượng (Object)	51
#43. Truy cập và cập nhật thuộc tính của Object	52
#44. Lặp Object sử dụng for...in và for...of	53
#45. Bài Tập Lab 03	54
#46. Chữa Bài Tập Lab 03	54
Chapter 6: DOM và Sự kiện (Event)	55
#47. Tổng quan về chapter 6	55
#48. DOM là gì? Giới thiệu cơ bản về DOM	55
#49. Truy cập phần tử HTML trong DOM	57
#50. Giới thiệu xử lý sự kiện (Events)	58
#51. Lắng nghe sự kiện với addEventListener	59
#52. Thay đổi nội dung của phần tử HTML	60
#53. Thay đổi CSS bằng JavaScript	61
#54. Alert (Extra)	62
#55. Local Storage (Extra)	63
#56. Bài Tập Lab 04	64
#57. Chữa Bài Tập Lab 04	64
Chapter 7: Xử Lý Bất Đồng Bộ (Asynchronous)	65
#58. Tổng quan về chapter 7	65
#59. Cài đặt môi trường Nodejs	65
#60. Setup Dự Án Backend	67
#61. Xử Lý Bất Đồng Bộ (Asynchronous) là gì ?	68
#62. Promise: Lời hứa từ tương lai	69
#63. Gọi API với Fetch	70
#64. Xử lý lỗi với try/catch/finally	71
#65. Callback và vấn đề Callback Hell	72
#66. Async/Await: Cú pháp cho code sạch đẹp	74
#67. Bài Tập Lab 05	75
#68. Chữa Bài Tập Lab 05	75
Chapter 8: Dự Án Thực Hành JavaScript	76
#69. Tổng quan về chapter 8	76
#70. Bài Tập Thực Hành 01	76
#71.1 Chữa Bài Tập Thực Hành 01 - Part 1	78
#71.2 Chữa Bài Tập Thực Hành 01 - Part 2	78
#71.3 Chữa Bài Tập Thực Hành 01 - Part 3	78
#72. Bài Tập Thực Hành 02	79
#73.1 Chữa Bài Tập Thực Hành 02 - Part 1	82
#73.2 Chữa Bài Tập Thực Hành 02 - Part 2	82
#73.3 Chữa Bài Tập Thực Hành 02 - Part 3	82
#74. What's next với JavaScript	83
Chapter 9: Giới thiệu TypeScript	84
#75. Tổng quan về chapter 9	84

#76. TypeScript là gì ?	84
#77. Tôi bắt buộc phải biết TypeScript không ?	85
#78. Cài đặt môi trường Nodejs (TypeScript)	86
#79. Quy Trình Chạy Dự Án TypeScript	87
#80. Cài đặt Typescript Compiler	88
#81. Hello world với Typescript (Basic)	89
#82. Setup Dự Án Thực Hành TypeScript (Node.js)	90
#83. Chạy dự án với HTML	92
#84. Giải Thích Cấu Trúc Dự Án Thực Hành (Extra)	93
#85. Tài liệu về TypeScript (Docs)	93
Chapter 10: Kiểu dữ liệu (Data Type) với TypeScript	94
#86. Tổng quan về chapter 10	94
#87. Vấn đề tồn đọng khi sử dụng JavaScript ?	94
#88. String, Number và Boolean	95
#89. Arrays	96
#90. any/unknown	97
#91. Type Annotations/Type Inference	98
#92. Functions	99
#93. Union Types	100
#94. Object Types	101
#95. Type Aliases	102
#96. Bài Tập Lab 06	103
#97. Chữa Bài Tập Lab 06	103
Chapter 11: Interface	104
#98. Tổng quan về chapter 11	104
#99. null và undefined	104
#100. Enum/Literal types (Extra)	105
#101. tuple	106
#102. Intersection Type	106
#103. Interface là gì	107
#104. Phân biệt Type Aliases và Interfaces	109
#105. Setup Dự Án TypeScript với HTML	110
#106. Type Assertions (Ép Kiểu Type)	111
#107. Sử dụng TypeScript với fetch API (Promise)	112
#108. Bài Tập Lab 07	113
#109. Chữa Bài Tập Lab 07	114
Chapter 12: Class và OOP (Object Oriented Programming)	115
#110. Tổng quan về chapter 12	115
#111. Lập Trình Hướng Đối Tượng Là Gì ? (OOP)	115
#112. Class và Object trong TypeScript	116
#113. Thuộc tính và Phương thức (Fields & Methods)	118
#114. Constructor – Hàm tạo	119
#115. Access Modifiers – Giới hạn truy cập	120
#116. Tính đóng gói (getter/setter) (Encapsulation)	121

#117. Kế thừa trong TypeScript (Inheritance)	122
#118. Interface và Implements	123
#119. Tính Đa Hình (Polymorphism)	124
#120. Tính Trừu Tượng - Abstract Class (Abstraction)	125
#121. Bài Tập Lab 08	126
#122. Chữa Bài Tập Lab 08	127
Chapter 13: Generic	128
#123. Tổng quan về chapter 13	128
#124. Giới thiệu về Generic	128
#125. Generic Functions (Hàm Generic)	129
#126. Sử dụng nhiều Type Variable	130
#127. Generic Types - Kiểu dữ liệu tổng quát	131
#128. Generic Classes (Lớp Generic)	132
#129. Generic Interfaces (Interface Generic)	133
#130. Generic Constraints (Ràng buộc Generic)	134
#131. Default Generic Type	135
#132. Các Utility Type có sẵn (Built-in)	136
Chapter 14: Module	137
#133. Module là gì ?	137
#134. Tạo module trong TypeScript	138
#135. Module Scope	138
#136. Named export vs Default export	139
#137. Import toàn bộ module	140
#138. So sánh CommonJS và ESM module	141
#139. Cấu hình module trong tsconfig.json	142
#140. Tổ chức dự án theo module	143
#141. Keyword Protected (Extra)	144
Chapter 15: Dự Án Thực Hành TypeScript (Project 03)	145
#142. Hướng Dẫn Tạo Dự Án Thực Hành 03	145
#143. Tích Hợp Bootstrap	146
#144. Hướng Dẫn Sử Dụng Component Bootstrap	147
#145. Hướng Dẫn Sử Dụng Modal	148
#146. Demo Dự án Thực Hành 03	149
#147. Bài Tập Thực Hành 03 - Part 1	150
#148. Chữa Bài Tập Thực Hành 03 - Part 1	151
#149. Bài Tập Thực Hành 03 - Part 2	151
#150. Chữa Bài Tập Thực Hành 03 - Part 2	151
#151. Bài Tập Thực Hành 03 - Part 3	152
#152. Chữa Bài Tập Thực Hành 03 - Part 3	152
#153. Bài Tập Thực Hành 03 - Part 4	153
#154. Chữa Bài Tập Thực Hành 03 - Part 4	154
Chapter 16: Dự Án Thực Hành TypeScript (Project 04)	155
#155. Demo Dự án Thực Hành 04	155
#156. Hướng Dẫn Tạo Dự Án Thực Hành 04	155

#157. Bài Tập Thực Hành 04 - Part 1	156
#158. Chữa Bài Tập Thực Hành 05 - Part 1	156
#159. Bài Tập Thực Hành 04 - Part 2	157
#160. Chữa Bài Tập Thực Hành 04 - Part 2	158
#161. Bài Tập Thực Hành 04 - Part 3	159
#162. Chữa Bài Tập Thực Hành 04 - Part 3	160
#163. Bài Tập Thực Hành 04 - Part 4	161
#164. Chữa Bài Tập Thực Hành 04 - Part 4	162
#165. Bài Tập Thực Hành 04 - Part 5	163
#166. Chữa Bài Tập Thực Hành 04 - Part 5	166
What's next với TypeScript	167
Đánh Giá (Review/Rating) Khóa Học	169

Chapter 1: Bắt buộc xem - Không bỏ qua chương học này

Hướng dẫn sử dụng khóa học hiệu quả, đạt chất lượng cao nhất

#1. Hướng Dẫn Sử Dụng Khóa Học Hiệu Quả

Bạn vui lòng "xem video lần lượt" theo trình tự. Vì khóa học như 1 dòng chảy, video sau sẽ kế thừa lại kết quả của video trước đó.

1. Dành cho học viên "có ít thời gian"

Nếu bạn vội, cần học nhanh, hoặc "bạn đã biết rồi", thì "vẫn xem video, cơ mà không cần code theo".

Lưu ý: vẫn xem qua tài liệu khóa học để biết "video hướng dẫn gì".

Đã "Không xem video", thì cần "đọc giáo án".

Có như vậy mới biết khóa học nó làm cái gì.

2. Dành cho học viên "thông thường"

Nguyên tắc:

- Xem video lần lượt
- Xem video kết hợp với giáo án. Bạn không cần take note, vì những điều quan trọng đã có trong giáo án

- Bạn vui lòng code theo video.

Nếu bạn "code theo ý bạn", vui lòng "không hỏi khi có bugs".

Câu chuyện này giống như việc bạn đi khám bệnh, nhưng không tin lời bác sĩ

=> Nếu bạn giỏi, bạn làm luôn bác sĩ, còn đi khám bệnh làm gì.

- Bạn có thể "code theo ý bạn muốn", sau khi "đã kết thúc khóa học"
- Nếu bạn có thắc mắc (hoặc có ý tưởng/nhận thấy bugs), take note lại, bạn hỏi, rồi mình giải đáp.

Chứ không phải là "tự ý làm theo điều các bạn muốn".

Vì đa phần, các bugs trong khóa học mình đã fix hết rồi.

Nên là yên tâm để học theo bạn nhé.

3. Về cách code

Bạn vui lòng code theo video, từ cách đặt tên biến, hàm. Vì mình đã tuân theo "convention tối thiểu" khi bạn đi làm đấy

4. Về bài tập thực hành

Đối với bài tập thực hành, bạn cứ code theo cách bạn hiểu, và kết hợp với "search Google, stackoverflow..."

Ưu tiên cách học thông qua các trang tài liệu chính thức (documents), search Google và đọc stackoverflow.

(đây là cách làm mình hướng dẫn trong khóa học)

Trường hợp đã thử mọi cách không có kết quả, đây là lúc sử dụng các công cụ AI:

- Hạn chế việc, copy/paste solution (tức là hỏi, lấy đáp án, và không hiểu logic thực hiện) từ các ứng dụng AI, ví dụ như ChatGPT
- Với beginners, cần phải xây nền móng thật vững trước khi sử dụng các ứng dụng AI. Nên nhớ, copy/paste nhưng phải hiểu bạn đang làm gì.
- **Nếu bạn sử dụng AI, nên thông qua việc hỏi - phản biện**, giống như việc bạn nhờ AI để thực hiện search/deep-research.
- Nên tự đặt câu hỏi là làm sao có thể code được như vậy, cần học những kiến thức gì, và quan trọng nhất, đừng quên bước thực hành.

Vì không có thực hành, tất cả chỉ là lý thuyết.

#2 + #3 + #4. Hướng Dẫn Truy Cập Tài Liệu Khóa Học

//todo

@hoidanit

#5. Về Quyền Tác Giả

Bản quyền bài giảng/khóa học (tác phẩm) thuộc về tác giả Hỏi Dân IT

Nghiêm cấm sao chép bài giảng dưới mọi hình thức khi chưa được sự cho phép của tác giả Hỏi Dân IT.

Mọi hành vi cố ý hoặc vô ý vi phạm, đều phải chịu trách nhiệm trước pháp luật.

Vì quyền lợi chính đáng của người học, Hỏi Dân IT (hoidanit.vn) rất mong nhận được sự hợp tác của tất cả các bạn.

Thông báo vi phạm, xin gửi về hòm thư: ads.hoidanit@gmail.com

hoặc inbox trực tiếp Facebook: <https://www.facebook.com/askITwithERIC/>

#7 + #8. Thông Tin Tác Giả Hỏi Dân IT

Về tác giả:

Mọi thông tin về Tác giả Hỏi Dân IT, các bạn có thể tìm kiếm tại đây:

Website chính thức: <https://hoidanit.vn/>

Youtube “Hỏi Dân IT” : <https://www.youtube.com/@hoidanit>

Tiktok “Hỏi Dân IT” : <https://www.tiktok.com/@hoidanit>

Fanpage “Hỏi Dân IT” : <https://www.facebook.com/askITwithERIC/>

Udemy Hỏi Dân IT: <https://www.udemy.com/user/eric-7039/>

Nếu bạn muốn nói chuyện với mình (giao lưu trao đổi võ công :), có thể xem mình livestream trực tiếp tối thứ 2 hàng tuần trên [Youtube Hỏi Dân IT](#)

Chapter 2: Giới thiệu và thiết lập môi trường thực hành

Cài đặt & chuẩn bị môi trường thực hiện dự án

#8. Tổng quan về chapter 2

//todo

#9. JavaScript là gì

1. Javascript là gì ?

JavaScript (JS) là một ngôn ngữ lập trình đa năng (multi-paradigm) được thiết kế ban đầu để tạo các trang web tương tác trên trình duyệt.

Ngày nay, JavaScript đã phát triển thành một ngôn ngữ toàn diện có thể chạy ở mọi nơi: từ trình duyệt đến máy chủ, ứng dụng desktop, mobile, và thậm chí IoT.

Một số đặc điểm chính:

- Ngôn ngữ thông dịch (interpreted) – Không cần biên dịch, có thể chạy ngay.
- Dạng script động – Có thể thay đổi nội dung HTML/CSS theo thời gian thực.
- Hỗ trợ lập trình hướng đối tượng (OOP), hàm (functional), mô-đun (modular).
- Là ngôn ngữ chính của Frontend Web và được hỗ trợ bởi mọi trình duyệt.

2. Ứng dụng thực tế của JavaScript

1. Phát triển Website (Frontend)

Dùng để điều khiển giao diện và trải nghiệm người dùng với thư viện/framework nổi tiếng như: React, Vue.js, Angular, Svelte

Ví dụ: Tạo Single Page Applications (SPA) như Facebook, Gmail...

2. Phát triển phía Server (Backend)

Dùng với Node.js để xây dựng server/API

Framework phổ biến: Express.js, NestJS, Fastify

3. Phát triển Mobile App

Sử dụng JavaScript để tạo ứng dụng di động đa nền tảng với: React Native, Ionic, Expo

4. Phát triển Game

Tạo game 2D/3D đơn giản với HTML5 + JS, dùng thư viện như Phaser, Three.js cho đồ họa

5. Ứng dụng Desktop

Viết app cho Windows/Mac/Linux bằng Electron.js

Ví dụ: Visual Studio Code, Slack, Discord đều viết bằng JavaScript

6. Ứng dụng AI, IoT, Automation

Xử lý mô hình học máy với TensorFlow.js

Giao tiếp thiết bị IoT qua JS runtime trên vi điều khiển (Espruino)

Viết script tự động thao tác trình duyệt (Puppeteer, Playwright)

#10. Chuyện cài đặt công cụ (Bắt Buộc Xem)

1. Mục đích

Chương học này sẽ hướng dẫn chi tiết cách cài đặt các công cụ cần thiết để phục vụ cho khóa học. Vì vậy, **bạn vui lòng không bỏ qua video nào, xem lần lượt theo thứ tự**

Có 2 sai lầm mà các bạn hay gặp phải, đặc biệt là những bạn “đã biết 1 chút”

Sai lầm 1: Bỏ qua các video cài đặt công cụ vì bạn cho rằng bạn “đã biết rồi”

Hãy nhớ rằng, **cài công cụ là 1 phần, đang còn phải “cấu hình” nó nữa.**

Khóa học được sinh ra, và đã tối ưu. Bạn chỉ dành 5 tới 10 phút để xem video, đổi lại tiết kiệm cho bạn cả giờ đồng hồ ngồi mò mẫm.

Sai lầm 2: Không quan tâm tới version của phần mềm

Khi thực hiện khóa học, **bạn vui lòng download và cài đặt version phần mềm giống như video. Điều này sẽ đảm bảo môi trường thực thi code là giống nhau. (hạn chế tối đa bug có thể xảy ra)**

2. Version Phần Mềm theo thời gian

Bạn đừng sợ phần mềm (công cụ) nó thay đổi version, hay thậm chí là “chê bai” version cũ. Vì vốn dĩ, công nghệ nó là vậy, luôn thay đổi theo thời gian.

Bạn yêu cầu “version mới nhất” cho cái bạn học, mình đã làm điều đó tại thời điểm quay video khóa học, tuy nhiên, sẽ là cố định 1 version.

Lý do: công nghệ sẽ cập nhật theo thời gian. Cho dù bạn muốn hay không, hoặc thậm chí lấp tên lửa vào đ*t, đuổi cũng không kịp.

Những cái ngày hôm nay, bạn cho là mới nhất, qua ngày mai, nó đã là cũ.

Điều bạn cần làm là: học 1 version, và quan trọng hơn, là bạn học, bạn cần hiểu nó

Sau đấy, nếu cần thiết, bạn học version mới hơn. Điểm khác biệt ở đây, là khi học version mới, bạn không phải là người bắt đầu từ số 0 (do đã có base từ version cũ)

Chỉ không học version cũ khi và chỉ khi: sản phẩm của version cũ không dùng được

Đấy là lý do tại sao các khóa học của mình, khi học xong, mình mới hướng dẫn nâng cấp version.

#11. Cài đặt & cấu hình Google Chrome

Ở đây, sử dụng google chrome vì nó là ứng dụng phổ biến nhất (trình duyệt web được dùng nhiều nhất)

Bạn nên dùng Google Chrome (thay vì Firefox/Edge...) để đảm bảo rằng thao tác sử dụng giữa bạn và mình là giống nhau (tránh gây khó khăn không cần thiết)

Lưu ý: sử dụng version tiếng anh
=> change language

Mục tiêu:

- Sử dụng Google Chrome để chạy ứng dụng web
 - Ngôn ngữ hiển thị là Tiếng Anh
 - Set default app là google chrome (nếu nó mở app, thì chạy với google chrome)
- Xem hướng dẫn setup default app cho windows [tại đây](#)

#12. Cài đặt môi trường code JavaScript

Editor/IDE (Integrated Development Environment) cần đáp ứng các tiêu chí:

- Giá cả hợp lý (ưu tiên mã nguồn mở - miễn phí)
- Dễ sử dụng
- Hỗ trợ cú pháp coding (gợi ý code, phát hiện lỗi)
- Hỗ trợ debug
- Cộng đồng sử dụng rộng rãi (community support)
- Hỗ trợ đa nền tảng: Windows, MacOS, Linux

1. Các công cụ phổ biến để code JavaScript:

Dùng miễn phí:

- [VSCode](#): rất phổ biến nhất
- [Sublime Text](#), [Atom](#)
- Notepad 📝

Dùng trả phí: [WebStorm](#) (JetBrains IDEs)

Trong khóa học này, mình sử dụng Visual Studio Code (VSCode), vì miễn phí, hỗ trợ tốt cho việc code JavaScript/TypeScript

2. Cài đặt Visual Studio Code (VSCode)

Link download: <https://code.visualstudio.com/download>

3. Cấu hình VSCode

Format Code

Setup Format on Save

Mục đích: Mỗi lần nhấn Ctrl + S , code sẽ được auto format trông cho đẹp/dễ nhìn

Các extensions cài đặt thêm:

- **Code Spell Checker** : hỗ trợ check chính tả khi đặt tên tiếng anh
- **Auto Complete Tag** : hỗ trợ code nhanh HTML

#13. Hello World với JavaScript

Bước 1: Tạo folder chứa source code

Lưu ý về tên dự án thực hành và đường dẫn (path) nơi lưu trữ code, không dùng ký tự đặc biệt và tiếng việt có dấu

Nên dùng: D://my-project/js-ts-hoidanit

Không nên dùng: D://dự án/học javascript

Đặt tên dự án thực hành là: **js-ts-hoidanit**

Bước 2: Tạo file html, js

//todo: viết hello world

Sử dụng [script tag](#) trong file html

Bước 3: Chạy với Google Chrome

//todo

#14. Sử dụng Git để quản lý mã nguồn

Mục đích: công cụ để quản lý mã nguồn và sử dụng source code của dự án (tránh tình trạng máy tính hư bị mất hết code)

Nếu bạn chưa biết gì về Git, xem nhanh [tại đây](#) (miễn phí)
Khóa học Git trả phí, tham khảo [tại đây](#)

- Sử dụng Git theo nguyên tắc:

1. Học xong video nào, commit đẩy lên Github/Gitlab

=> tạo cơ hội để thực hành câu lệnh của Git, ví dụ:

git add

git commit

git push...

2. Git là công cụ "mặc định bạn phải biết" khi đi làm phần mềm

=> điều 1 ở trên giúp bạn thực hành

3. Thói quen học xong video nào, đẩy code lên Git, giúp bạn tạo ra bản "backup" cho project của bạn

Ví dụ máy tính bạn bị hỏng đột xuất/bị mất

=> vẫn còn code, chỉ cần pull về code tiếp mà không phải code từ đầu.

=> Mục đích sử dụng git ở đây là : backup code + thực hành công cụ đi làm mà bạn "phải biết" nếu muốn đi thực tập/đi làm.

Chapter 3: Biến, Kiểu dữ liệu và Toán tử

Tìm hiểu và nắm vững cách khai báo cú pháp cơ bản và việc sử dụng biến số của JavaScript

#15. Tổng quan về chapter 3

//todo

#16. Google Chrome Devtool (Extra)

Google Chrome DevTools (Developer Tools) là bộ công cụ được tích hợp sẵn trong trình duyệt Chrome, cho phép lập trình viên kiểm tra, sửa lỗi, phân tích hiệu năng, và tối ưu hóa website hoặc ứng dụng web ngay trong trình duyệt.

Lưu ý: bạn nên sử dụng Google Chrome, thay vì FireFox, Opera, Cốc Cốc... để đảm bảo thao tác của bạn và video là giống nhau.

Cách mở Chrome DevTools:

Nhấn **F12** hoặc **Ctrl + Shift + I** (Windows/Linux) / **Cmd + Option + I** (Mac)

Hoặc: Click chuột phải vào một phần tử trên trang → chọn “Inspect” (Kiểm tra)

Các tab chính của DevTools:

1. **Elements** : hiển thị DOM Tree và CSS đang có trên website
2. **Console**: cung cấp giao diện thực thi lệnh JavaScript trực tiếp, hiển thị các lỗi JavaScript.
3. **Sources**: hiển thị cấu trúc file JS/CSS..., hỗ trợ đặt breakpoint để debug.
4. **Network**: theo dõi tất cả các request được gửi từ trình duyệt

5. Application: quản lý dữ liệu lưu trữ:

Local Storage / Session Storage/ Cookies

IndexedDB/ Cache / Service Worker

6. Performance

Đo hiệu năng trang web: Thời gian load, render, paint.

Phân tích các thao tác nặng gây giật, delay.

Sử dụng để tối ưu frontend.

#17. Biến trong JavaScript (var, let, const)

//todo: tạo thư mục src để lưu source code theo từng bài học

Để khai báo biến (variable), cú pháp chung là **<Từ khóa> <tên biến> = <giá trị>;**

Ví dụ:

```
var name = "Alice";
```

```
let age = 25;
```

```
const country = "Vietnam";
```

Lịch sử ra đời:

https://www.w3schools.com/js/js_versions.asp

Phân biệt nhanh [var/let](#)

Yêu cầu bắt buộc:

- Ưu tiên dùng **let** và **const** thay vì **var**.
- Dùng **const** khi giá trị không cần thay đổi.

#18. Kiểu dữ liệu nguyên thủy (Primitive Data Types)

Mục tiêu:

- Hiểu được các kiểu dữ liệu nguyên thủy trong JavaScript
- Biết cách sử dụng typeof để kiểm tra kiểu dữ liệu

Định nghĩa: Kiểu dữ liệu nguyên thủy (primitive) là những kiểu dữ liệu cơ bản, không có phương thức hay thuộc tính, và được lưu trực tiếp giá trị trong vùng nhớ của biến

7 kiểu dữ liệu nguyên thủy trong JavaScript

Kiểu dữ liệu	Ví dụ giá trị	Ghi chú
Number	10; 3.14	Gồm cả số nguyên và số thực
String	hoidanit	Chuỗi ký tự đặt trong nháy đơn hoặc nháy kép
Boolean	true, false	Chỉ có 2 giá trị
Undefined	undefined	Khai báo biến số, và không gán giá trị cho biến số
Null	null	Khi khai báo biến, giá trị bằng null. Âm chỉ rằng, giá trị của biến số "nothing"/"empty"
BigInt	123456789n	Dùng cho số rất lớn (thêm n ở cuối), tối đa $2^{53} - 1$
Symbol	Symbol("id")	Dùng để tạo giá trị duy nhất

Lưu ý:

JavaScript không phân biệt số nguyên hay số thực, chỉ gọi chung là number

Có thể kiểm tra **datatype** bằng cách sử dụng: typeof

```
console.log(typeof 10);    // "number"
```

```
console.log(typeof "hello"); // "string"
```

#19. Kiểu dữ liệu tham chiếu (Object Data Types)

Định nghĩa: Dữ liệu không được lưu trực tiếp trong biến, mà biến chỉ chứa một địa chỉ tham chiếu trong bộ nhớ.

Tên kiểu dữ liệu	Ví dụ	Ghi chú
Object	{ name: "hoidanit" }	Dạng key-value, linh hoạt
Array	[1, 2, 3]	Danh sách các phần tử
Function	function() {}	
Date	new Date()	Xử lý thời gian, ngày tháng
RegExp	/abc/	Biểu thức chính quy
Map	new Map()	Dạng key-value
Set	new Set()	Lưu giá trị duy nhất, không trùng lặp

Tập trung vào 2 loại data chính: Object và Array

#20. Toán tử trong JavaScript

1. Toán tử số học (Arithmetic Operators)

Toán tử	Ý nghĩa	Ví dụ	Kết quả
+	Cộng	5 + 2	7
-	Trừ	5 - 2	3
*	Nhân	5 * 2	10
/	Chia	10 / 2	5
%	Chia lấy dư	5 % 2	1
**	Lũy thừa	2 ** 3	8
++	Tăng 1	let a=10; a++	11
--	Giảm 1	let a=10; a--	9

//so sánh a++ và ++a. Lấy ví dụ

2. Toán tử so sánh (Comparison Operators)

Toán tử	Ý nghĩa	Ví dụ	Kết quả
>	Lớn hơn	5 > 3	true
<	Nhỏ hơn	2 < 1	false
>=	Lớn hơn hoặc bằng	5 >= 5	true
<=	Nhỏ hơn hoặc bằng	4 <= 3	false
==	So sánh bằng (lỏng)	5 == '5'	true
===	So sánh bằng (chặt)	5 === '5'	false
!=	Khác nhau (lỏng)	5 != '5'	false
!==	Khác nhau (chặt)	5 !== '5'	true

Luôn dùng === và !== nếu muốn so sánh datatype (kiểu) và giá trị (value)

Ngoài ra, còn có thể sử dụng:

Toán tử logic

```
let isAdult = true;
```

```
let hasID = false;
```

```
console.log(isAdult && hasID); // false : toán tử và (AND)
```

```
console.log(isAdult || hasID); // true : toán tử hoặc (OR)
```

```
console.log(!hasID); // true: toán tử khác/phủ định (NOT)
```

#21. Câu lệnh console trong JavaScript

//todo: hướng dẫn cách comment code

Console là câu lệnh đã được khai báo sẵn trong môi trường javascript (tương tự bạn có hàm printf khi học Java/C)

1. Các câu lệnh thường gặp

console.log() – In giá trị ra màn hình console

console.error() – Hiển thị lỗi

console.warn() – Hiển thị cảnh báo

//phân biệt error và warning

2. Các tip hay dùng

Sử dụng dấu phẩy thay vì cộng chuỗi trong console.log()

```
let name = "hoidanit";
```

```
let age = 20;
```

// Không nên: cộng chuỗi dễ sai

```
console.log("Name: " + name + ", Age: " + age);
```

// **Nên: dùng dấu phẩy** (tự động thêm dấu cách)

```
console.log("Name:", name, "Age:", age);
```

Tô màu cho console.log() bằng %c

```
console.log("%cCảnh báo!", "color: red; font-weight: bold;");
```

#22. Template String (Extra)

Tài liệu:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals

1. Template String là gì?

Template String (còn gọi là Template Literals) là cách khai báo chuỗi bằng **dấu backtick** (```) thay vì nháy đơn `'` hay nháy kép `"`.

Chúng hỗ trợ:

- Chèn biến trực tiếp bằng `${}`
- Xuống dòng không cần `\n`
- Chèn biểu thức JS vào trong chuỗi

Cách code cũ:

```
let name = "hoidanit";  
let age = 20;  
console.log("Tên: " + name + ", Tuổi: " + age);
```

Sử dụng template string:

```
console.log(`Tên: ${name}, Tuổi: ${age}`);
```

#23. Bài Tập Lab 01

Yêu cầu:

Tạo các biến sau với const hoặc let (chọn phù hợp):

fullName: tên đầy đủ (string)

birthYear: năm sinh (number)

isStudent: true/false

Tính tuổi hiện tại dựa trên birthYear.

Gợi ý: [tính năm](#) hiện tại, sử dụng:

```
const today = new Date();  
const currentYear = today.getFullYear();
```

In ra console theo format:

Tên: [fullName]

Tuổi: [calculatedAge]

Sinh viên: [Đúng/Sai]

#24. Chữa Bài Tập Lab 01

//todo

Chapter 4: Cấu Trúc Điều Khiển & Hàm (Function)

Sử dụng câu điều kiện *if-else*, *switch-case*, vòng lặp *for/while* và *function* với JavaScript

#25. Tổng quan về chapter 4

//todo

#26. Câu lệnh điều kiện – if / else / else if

//todo: chia source code theo tên chapter

1. Khái niệm

Câu lệnh điều kiện cho phép chương trình ra quyết định. Tùy thuộc vào điều kiện đúng hoặc sai, ta có thể thực hiện những đoạn mã khác nhau.

Cú pháp:

```
if (điều_kiện) {  
    // khối lệnh nếu điều_kiện đúng (true)  
}
```

Hoặc:

```
if (điều_kiện) {  
    // nếu đúng  
} else {  
    // nếu sai  
}
```

#27. Câu lệnh switch / case

switch là một cấu trúc điều kiện giúp kiểm tra giá trị của một biểu thức và thực thi các khối lệnh tương ứng với từng **case (trường hợp cụ thể)**.

Sử dụng **switch** giúp mã nguồn **gọn gàng hơn** khi cần kiểm tra nhiều giá trị khác nhau của cùng một biến.

Cú pháp:

```
switch (biểu_thức) {  
  case giá_trị_1:  
    // khối lệnh nếu biểu_thức === giá_trị_1  
    break;  
  
  case giá_trị_2:  
    // khối lệnh nếu biểu_thức === giá_trị_2  
    break;  
  
  ...  
  
  default:  
    // khối lệnh nếu không khớp bất kỳ case nào  
}
```

#28. Vòng lặp (for, while, do...while)

Vòng lặp giúp **tự động hóa việc lặp lại các khối lệnh**, thay vì phải viết lặp thủ công nhiều lần. Chỉ cần thiết lập điều kiện và hành động lặp lại là xong.

1. Vòng lặp for

Cú pháp:

```
for (khởi_tạo; điều_kiện; cập_nhật) {  
    // khối lệnh lặp  
}
```

- khởi_tạo: Gán giá trị ban đầu cho biến đếm.
- điều_kiện: Kiểm tra trước mỗi vòng lặp. Nếu đúng thì chạy tiếp.
- cập_nhật: Tăng/giảm biến đếm.

2. Vòng lặp while

Cú pháp:

```
while (điều_kiện) {  
    // khối lệnh lặp  
}
```

- Kiểm tra điều kiện trước mỗi vòng lặp.
- Nếu điều kiện đúng thì tiếp tục chạy, sai thì dừng.

3. Vòng lặp do...while

Cú pháp:

```
do {  
    // khối lệnh lặp  
} while (điều_kiện);
```

Chạy ít nhất 1 lần, sau đó mới kiểm tra điều kiện.

//todo:

So sánh vòng lặp while và do...while ?

#29. Câu lệnh điều khiển luồng – break, continue

Trong quá trình lập, đôi khi bạn không muốn thực hiện toàn bộ số vòng lặp, mà chỉ muốn:

- Thoát ra hoàn toàn khỏi vòng lặp → dùng **break**
- Bỏ qua vòng hiện tại và tiếp tục vòng sau → dùng **continue**

1. Câu lệnh break

Dùng để **thoát khỏi vòng lặp ngay lập tức**, kể cả khi điều kiện chưa sai.

Ví dụ:

```
for (let i = 1; i <= 10; i++) {  
  if (i === 5) break;  
  console.log(i);  
}
```

2. Câu lệnh continue

Dùng để **bỏ qua vòng lặp hiện tại và chuyển sang vòng tiếp theo**.

Khối lệnh sau continue sẽ không được thực thi trong vòng hiện tại.

Ví dụ:

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue;  
  console.log(i);  
}
```

#30. Hàm (Function) trong JavaScript

1. Hàm là gì?

Hàm là một **khối mã có tên**, thực hiện một công việc cụ thể.

Hàm giúp **tái sử dụng mã code**, chia chương trình thành các phần nhỏ để quản lý hơn.

Cú pháp:

```
function tenHam() {
```

```
    // Khối lệnh
```

```
}
```

Ví dụ:

```
function greet() {
```

```
    console.log("Xin chào!");
```

```
}
```

`greet();` // Output: Xin chào!

//Gọi hàm bằng cách dùng tên hàm kèm dấu ngoặc ()

2. Tham số và đối số (Parameters & Arguments)

```
function greet(name) {
```

```
    console.log("Xin chào, " + name + "!");
```

```
}
```

`greet("hoidanit");` // Output: Xin chào, hoidanit!

- **Tham số:** biến khai báo trong hàm (name).
- **Đối số:** giá trị truyền vào khi gọi hàm ("hoidanit").

#31. Arrow function (Extra)

Tài liệu:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

//todo

Về Immediately Invoked Function Expression / anonymous function:

<https://developer.mozilla.org/en-US/docs/Glossary/IIFE>

#32. Keyword Return

1. return là gì?

return là từ khóa dùng để **kết thúc hàm và trả về một giá trị** ra ngoài.

Khi JavaScript gặp **return**, hàm sẽ **dừng thực thi ngay lập tức**.

Cú pháp:

```
function tenHam() {  
  return giaTriTraVe;  
}
```

// nếu không return giá trị thì sao ?

#33. Phạm vi biến (Scope)

1. Scope là gì?

Scope là phạm vi truy cập của biến – tức là nơi biến đó có thể được sử dụng.

Trong JavaScript, có 3 loại phạm vi chính:

- Phạm vi toàn cục (**Global Scope**)
- Phạm vi hàm (**Function Scope**)
- Phạm vi khối (**Block Scope**)

2. Phạm vi toàn cục – Global Scope

Biến được khai báo ngoài mọi hàm hoặc khối lệnh. Có thể **truy cập từ bất kỳ đâu** trong chương trình.

Ví dụ:

```
let globalVar = "Tôi là biến toàn cục";  
function show() {  
  console.log(globalVar); // Truy cập được  
}  
show();  
console.log(globalVar); // Truy cập được
```

3. Phạm vi hàm – Function Scope

Biến khai báo **bên trong một hàm** chỉ dùng được **bên trong hàm đó**.

Ví dụ:

```
function sayHi() {  
  let name = "hoidanit";  
  console.log("Hi " + name);  
}
```

```
sayHi();  
console.log(name); // ❌ Lỗi: name is not defined
```

4. Phạm vi khối – Block Scope (ES6 trở lên)

Biến khai báo bằng **let** và **const** trong cặp dấu {} chỉ dùng được bên trong khối đó.

Ví dụ:

```
if (true) {  
  let x = 10;  
  const y = 20;  
  console.log(x, y); //  Truy cập được  
}
```

```
console.log(x); //  Lỗi: x is not defined
```

#34. Bài Tập Lab 02

Yêu cầu:

1. Tạo hàm **tinhTrungBinh**(toan, van, anh)

- Hàm nhận vào 3 điểm số (sử dụng tham số).
- Trả về điểm trung bình (sử dụng return).

2. Tạo hàm **xepLoai**(diemTB)

Dựa vào điểm trung bình, xếp loại học sinh:

Từ 9 → "Xuất sắc"

Từ 8 và nhỏ hơn 9 → "Giỏi"

Từ 6.5 và nhỏ hơn 8 → "Khá"

Còn lại → "Trung bình"

Dùng if / else if / else để thực hiện

Output:

Cho trước 3 biến:

```
const diemToan = 9;
```

```
const diemVan = 8;
```

```
const diemAnh = 7;
```

Gọi các hàm trên để tính điểm trung bình và in ra kết quả sau:

Điểm trung bình: 8.0

Xếp loại: Giỏi

#35. Chữa Bài Tập Lab 02

```
//todo
```

Chapter 5: Mảng (Array) và Đối Tượng (Object)

Làm việc với các dạng dữ liệu phức tạp, bao gồm xử lý array và object

#36. Tổng quan về chapter 5

//todo

#37. Giới thiệu về Mảng (Array)

1. Mảng là gì?

Mảng (array) là một kiểu dữ liệu đặc biệt trong JavaScript, cho phép bạn **lưu nhiều giá trị trong một biến duy nhất**.

Ví dụ: thay vì sử dụng:

```
let color1 = "red";  
let color2 = "green";  
let color3 = "blue";
```

//sử dụng array: let colors = ["red", "green", "blue"];

2. Cách khai báo mảng

Dùng dấu ngoặc vuông []

Ví dụ:

```
let fruits = ["apple", "banana", "orange"];
```

JavaScript **không giới hạn kiểu dữ liệu** bên trong mảng. Bạn có thể lưu chuỗi, số, boolean, object, thậm chí cả mảng khác.

Ví dụ:

```
let mixed = [42, "hello", true, null, [1, 2, 3]];
```

#38. Truy cập, chỉnh sửa, thêm và xóa phần tử của mảng

1. Truy cập phần tử trong mảng

Để truy cập một phần tử, bạn sử dụng cú pháp: **array[index]**

Lưu ý: Chỉ số (index) bắt đầu từ 0

Ví dụ:

```
let colors = ["red", "green", "blue"];
```

```
console.log(colors[0]); // "red" (phần tử đầu tiên)
```

```
console.log(colors[1]); // "green"
```

```
console.log(colors[2]); // "blue"
```

Nếu truy cập phần tử không tồn tại → kết quả là **undefined**

2. Chỉnh sửa phần tử trong mảng

Ví dụ:

```
let fruits = ["apple", "banana", "orange"];
```

```
fruits[1] = "grape"; // Thay "banana" bằng "grape"
```

```
console.log(fruits); // ["apple", "grape", "orange"]
```

3. Thêm, xóa

Thêm phần tử:

vào cuối mảng với **push()**

vào đầu mảng với **unshift()**

Xóa phần tử:

cuối mảng với **pop()**

đầu mảng với **shift()**

#39. Duyệt mảng với vòng lặp và forEach

1. Duyệt mảng bằng vòng lặp for

Ví dụ:

```
let scores = [80, 90, 70];
```

```
for (let i = 0; i < scores.length; i++) {  
  console.log("Phần tử thứ " + i + ": " + scores[i]);  
}
```

2. Duyệt bằng forEach()

```
let names = ["Alice", "Bob", "Charlie"];
```

```
names.forEach(function(name, index) {  
  console.log(index + ": " + name);  
});
```

//todo: sử dụng arrow function

#40. Biến đổi mảng với map()

Tài liệu:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map

1.map() là gì?

Phương thức **map()** được dùng để duyệt qua từng phần tử trong mảng, **tạo ra một mảng mới** mà mỗi phần tử là kết quả biến đổi của phần tử tương ứng trong mảng ban đầu.

map() không thay đổi mảng gốc.

Cú pháp:

```
let newArray = array.map(function(element, index, array) {  
  // return giá trị mới  
});
```

element: phần tử hiện tại

index (tùy chọn): chỉ số phần tử

array (tùy chọn): chính mảng đang được duyệt

return: chính là phần tử mới của mảng kết quả

//todo: sử dụng arrow function

#41. Lọc phần tử mảng với filter()

Tài liệu:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter

1. filter() là gì?

filter() là một phương thức của mảng, cho phép bạn **lọc ra các phần tử thỏa mãn điều kiện**, và trả về một mảng mới chỉ chứa các phần tử đó.

Mảng gốc không bị thay đổi

Cú pháp:

```
let newArray = array.filter(function(element, index, array) {  
  return điều_kiện_đúng; // true giữ lại, false loại bỏ  
});
```

element: phần tử hiện tại

index (tùy chọn): chỉ số phần tử

array (tùy chọn): mảng gốc đang được duyệt

//todo: sử dụng arrow function

#42. Giới thiệu về Đối tượng (Object)

1. Đối tượng là gì?

Trong JavaScript, **đối tượng (object)** là kiểu dữ liệu dùng để **lưu trữ thông tin có cấu trúc** dưới dạng cặp tên **thuộc tính** (key) và **giá trị** (value).

Nếu mảng lưu danh sách giá trị, thì đối tượng lưu thông tin mô tả rõ từng giá trị.

Mảng (Array)	Đối tượng (Object)
["hoidanit", 25]	{ name: "hoidanit", age: 25 }
Dùng chỉ số (index)	Dùng tên thuộc tính (key)

2. Cách khai báo đối tượng

```
let obj_name = {  
  key: value  
};
```

#43. Truy cập và cập nhật thuộc tính của Object

1. Truy cập thuộc tính trong Object

Cách 1: Dùng dấu chấm (.)

```
let user = {  
  name: "hoidanit",  
  age: 25  
};
```

```
console.log(user.name); // "hoidanit"  
console.log(user.age); // 25
```

Cách 2: Dùng dấu ngoặc vuông ([])

```
console.log(user["name"]); // "hoidanit"
```

2. Cập nhật và thêm thuộc tính

```
user.age = 30;           // Cập nhật  
user["city"] = "Hanoi"; // Thêm mới
```

```
delete user.age; //xóa thuộc tính
```

#44. Lặp Object sử dụng for...in và for...of

for...in duyệt qua thuộc tính

for...of duyệt qua giá trị

Cú pháp:

```
for (let key in object) {  
  console.log(key, object[key]);  
}
```

```
for (let value of iterable) {  
  console.log(value);  
}
```

#45. Bài Tập Lab 03

Yêu cầu:

Hãy tạo một **mảng products** chứa danh sách 5 sản phẩm, mỗi sản phẩm là một **object** có các thuộc tính:

- **name** (tên sản phẩm)
- **price** (giá sản phẩm)
- **inStock** (true/false – còn hàng hay không)

Ví dụ:

```
{  
  name: "T-shirt",  
  price: 200,  
  inStock: true  
}
```

Hãy thực hiện:

1. In ra **tên của sản phẩm đầu tiên**.
2. **Thay đổi giá sản phẩm thứ hai** thành 150 và in ra danh sách tất cả sản phẩm
3. **Thêm một sản phẩm mới** vào cuối mảng và in ra danh sách tất cả sản phẩm
4. **Xoá sản phẩm cuối cùng** ra khỏi danh sách và in ra danh sách tất cả sản phẩm
5. Dùng **forEach()** để in ra tất cả **tên sản phẩm**.
6. Dùng **map()** để tạo mảng mới chỉ chứa **giá sản phẩm**.
7. Dùng **filter()** để lấy các **sản phẩm còn hàng** (inStock = true).
8. Dùng **for...in** để duyệt qua thuộc tính của sản phẩm đầu tiên.

#46. Chữa Bài Tập Lab 03

```
//todo
```

Chapter 6: DOM và Sự kiện (Event)

Hiểu và thao tác với DOM trong JavaScript. Biết cách truy cập, thay đổi nội dung HTML và xử lý sự kiện tương tác bằng JavaScript

#47. Tổng quan về chapter 6

//todo

#48. DOM là gì? Giới thiệu cơ bản về DOM

1. DOM là gì?

DOM (viết tắt của Document Object Model) là một mô hình dạng cây đại diện cho cấu trúc của một trang web.

- Khi trình duyệt tải trang HTML, nó phân tích (parse) nội dung HTML và tạo ra một mô hình dạng cây DOM.
- Mỗi thẻ HTML trong tài liệu sẽ được chuyển thành một "đối tượng" (object) mà JavaScript có thể truy cập và thao tác.

DOM trông như thế nào?

Giả sử bạn có đoạn HTML sau:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Trang web của hoidanit</title>
  </head>
  <body>
    <h1>Chào bạn!</h1>
    <p>Đây là một đoạn văn bản.</p>
  </body>
</html>
```

DOM sẽ biểu diễn như một cây phân cấp:

```
Document
├── html
│   ├── head
│   │   └── title
│   └── body
│       ├── h1
│       └── p
```


2. DOM cho phép làm gì?

Với DOM, bạn có thể dùng JavaScript để:

- ✓ Truy cập và đọc nội dung của các phần tử HTML
- ✓ Thay đổi nội dung hoặc thay đổi kiểu hiển thị (CSS)
- ✓ Thêm, xóa phần tử HTML
- ✓ Lắng nghe sự kiện người dùng: click chuột, nhập dữ liệu, rê chuột, v.v.

#49. Truy cập phần tử HTML trong DOM

Khi bạn muốn thay đổi nội dung, thêm màu sắc, ẩn/hiện một phần tử, hoặc xử lý khi người dùng tương tác (như click, nhập dữ liệu...), bạn cần lấy được phần tử HTML đó thông qua DOM.

Một số cách phổ biến để truy cập phần tử

✓ **document.getElementById(id)**

Dùng để lấy phần tử có id cụ thể.

Trả về duy nhất một phần tử (hoặc null nếu không tìm thấy).

html:

```
<p id="intro">Chào mừng!</p>
```

js:

```
const para = document.getElementById("intro");  
console.log(para); // in ra phần tử <p>
```

✓ **document.querySelector(selector)**

Dùng để **tìm phần tử đầu tiên khớp** với CSS selector (id, class, tag, v.v.).

```
const firstNote = document.querySelector(".note");  
console.log(firstNote); // chỉ in ra phần tử đầu tiên có class="note"
```

✓ **document.querySelectorAll(selector)**

Dùng để **lấy tất cả phần tử khớp với selector**, trả về NodeList (giống mảng).

```
const allNotes = document.querySelectorAll(".note");  
console.log(allNotes.length);
```

#50. Giới thiệu xử lý sự kiện (Events)

Tài liệu:

<https://developer.mozilla.org/en-US/docs/Web/Events>

1. Sự kiện là gì?

Sự kiện (event) là hành động do người dùng hoặc trình duyệt tạo ra khi tương tác với trang web, ví dụ click, cuộn chuột...

=> Khi sự kiện xảy ra, JavaScript có thể thực hiện một đoạn mã để phản hồi lại sự kiện đó.

Một số event thường gặp:

click Người dùng nhấn chuột

input Khi người dùng gõ vào ô nhập liệu

change Khi giá trị input hoặc select thay đổi

keydown Khi nhấn phím bất kỳ trên bàn phím

submit Khi gửi form

Gắn sự kiện trực tiếp trong HTML (cách làm basic)

```
<button onclick="sayHello()">Chào</button>
```

```
<script>
function sayHello() {
  alert("Xin chào!");
}
</script>
```

Ưu điểm: Dễ viết, dễ hiểu cho người mới.

Nhược điểm: Không tách riêng JavaScript và HTML, khó bảo trì khi trang web lớn.

#51. Lắng nghe sự kiện với `addEventListener`

Tài liệu:

<https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener>

1. `addEventListener()` là gì?

`addEventListener()` là phương thức tiêu chuẩn và hiện đại để gắn sự kiện vào phần tử HTML.

Khi sự kiện xảy ra (người dùng click, nhập, rê chuột...), trình duyệt sẽ gọi hàm xử lý bạn đã chỉ định.

Cú pháp: `element.addEventListener("eventName", function);`

- **element:** phần tử HTML bạn muốn gắn sự kiện.
- **"eventName":** tên sự kiện, ví dụ "click", "input", "mouseover".
- **function:** hàm xử lý khi sự kiện xảy ra

Ví dụ:

```
<button id="myBtn">Bấm vào đây</button>
```

```
<script>
```

```
  const button = document.getElementById("myBtn");
```

```
  button.addEventListener("click", function () {
```

```
    alert("Bạn vừa nhấn nút!");
```

```
  });
```

```
</script>
```

#52. Thay đổi nội dung của phần tử HTML

innerText cho phép bạn lấy hoặc thay đổi phần nội dung dạng chữ mà người dùng thực sự nhìn thấy trên trang web (có tính đến CSS)

Ví dụ:

```
<p id="intro">Chào bạn!</p>
```

```
<script>
  const p = document.getElementById("intro");
  p.innerText = "Chào mừng đến với khóa học @hoidanit!";
</script>
```

innerHTML cho phép bạn lấy hoặc thay đổi nội dung HTML bên trong phần tử, bao gồm cả thẻ lồng bên trong.

```
<p id="greeting">Xin chào!</p>
```

```
<script>
  const p = document.getElementById("greeting");
  p.innerHTML = "<strong>Chào bạn,</strong> <em>học lập trình với hoidanit!</em>";
</script>
```

#53. Thay đổi CSS bằng JavaScript

1. Sử dụng style

Mỗi phần tử HTML trong DOM đều có thuộc tính `.style` để bạn thay đổi trực tiếp các thuộc tính CSS.

Cú pháp:

`element.style.tênThuộcTÍNHCSS = "giá trị mới";`

Lưu ý: Tên thuộc tính CSS dạng kebab-case như `background-color` sẽ được chuyển sang `camelCase` trong JavaScript: `backgroundColor`.

Ví dụ:

```
<p id="title">Xin chào!</p>
<button onclick="changeColor()">Đổi màu chữ</button>

<script>
  function changeColor() {
    const p = document.getElementById("title");
    p.style.color = "red";
  }
</script>
```

2. Sử dụng classList

Thêm hoặc xóa class bằng `.classList`

Thay vì thay đổi từng thuộc tính `.style`, bạn có thể thêm class CSS định nghĩa sẵn.

```
element.classList.add("ten-class");
element.classList.remove("ten-class");
```

#54. Alert (Extra)

1.Alert là gì?

alert() là **hàm có sẵn** trong JavaScript dùng để **hiển thị một hộp thoại thông báo** đơn giản (popup) cho người dùng.

Khi xuất hiện, hộp thoại sẽ **chặn toàn bộ trang web** cho đến khi người dùng nhấn nút "OK".

Ví dụ:

```
alert("Nội dung bạn muốn hiển thị");
```

#55. Local Storage (Extra)

1. Local Storage là gì?

Local **Storage** là một phần của Web Storage API, cho phép bạn lưu trữ dữ liệu ngay trong trình duyệt của người dùng.

Đặc điểm:

- **Dữ liệu không bị mất** khi reload hoặc tắt trình duyệt.
- Lưu trữ dạng **key-value** (khóa-giá trị), kiểu chuỗi (string).

Cú pháp:

Phương thức	Mô tả
localStorage. setItem (key, value)	Lưu dữ liệu (value phải là chuỗi)
localStorage. getItem (key)	Lấy dữ liệu theo key
localStorage. removeItem (key)	Xóa dữ liệu theo key
localStorage. clear ()	Xóa toàn bộ dữ liệu đã lưu trong Local Storage

//todo: convert object => json (**stringify/parse**)

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/parse

#56. Bài Tập Lab 04

Yêu cầu: Tạo form đăng nhập đơn giản

Tạo một **form html** gồm:

- **Input nhập username**
- **Input nhập password**
- **Button "Đăng nhập"**

Khi người dùng nhấn nút "Đăng nhập":

- Lấy giá trị từ hai ô nhập liệu
- So sánh với username/password đã định nghĩa sẵn trong JavaScript (hardcode)
username: hoidanit@gmail.com
password: 123456

Nếu đúng:

- Hiện thông báo "Đăng nhập thành công!" bằng alert()
- **Redirect** tới trang html (success.html)

Nếu sai:

- Hiện alert("Tài khoản hoặc mật khẩu sai")
- Đổi màu viền ô input thành đỏ (dùng .style.borderColor)

#57. Chữa Bài Tập Lab 04

//todo

Chapter 7: Xử Lý Bất Đồng Bộ (Asynchronous)

Sử dụng Callback, Promise và async await để xử lý bất đồng bộ trong javascript

#58. Tổng quan về chapter 7

//todo

#59. Cài đặt môi trường Nodejs

Mục đích: môi trường để chạy code Javascript ở phía backend (server)

Tài liệu: <https://nodejs.org/en>

1. Nodejs là gì ?

Nodejs không phải là thư viện (library), không phải framework của JavaScript.

Nodejs là môi trường để bạn thực thi code javascript, tại browser và server.

Bạn học Node.js, về bản chất, là học các thư viện/framework (viết bằng JavaScript), nên bạn cần cài đặt môi trường Nodejs để có thể thực thi code JavaScript

Điều này tương tự với:

Bạn học cách sử dụng Microsoft Excel (javascript)

Bạn cần cài hệ điều hành Windows để có thể học nó (nodejs)

2. Cài đặt Nodejs

Sai lầm của beginners, là không quan tâm tới version của phần mềm. Nên nhớ, công nghệ nó thay đổi theo thời gian, vì vậy, để hạn chế tối đa lỗi tối đa, bạn nên dùng version phần mềm như khóa học hướng dẫn.

Điều này tương tự với:

Bạn đang chơi 1 con game rất ngon trên Windows 7, bạn vác lên Windows 10 để chạy, có điều gì để đảm bảo rằng "sẽ không có lỗi xảy ra" ?

Trong khóa học này, mình sử dụng **version Node.js là 22.13.0**

Vì vậy, để hạn chế tối đa lỗi có thể xảy ra, bạn vui lòng cài đặt chính xác version nodejs ở trên

Khi code giống nhau, môi trường thực thi code giống nhau (version nodejs), thì rất hiếm khi lỗi xảy ra.

Nếu đây là lần đầu tiên bạn học (coding) một dự án với Node.js, mình khuyến khích sử dụng duy nhất 01 version Node.js (dễ quản lý)

Chỉ sử dụng nhiều version Node.js, khi và chỉ khi, trên bạn có nhiều dự án Node.js, và mỗi dự án yêu cầu một version Node.js khác nhau. (hướng dẫn tại mục 3 bên dưới)

Link tải nodejs v22.13.0:

<https://nodejs.org/download/release/v22.13.0/>

Sau khi cài đặt xong, kiểm tra bằng cách gõ câu lệnh:

node -v

3. Trường hợp dùng nhiều version Nodejs

Lưu ý: bạn cần gỡ nodejs trước khi cài nvm

//áp dụng cho windows

<https://github.com/coreybutler/nvm-windows>

//áp dụng cho macos

Video hướng dẫn cài nvm cho mac, xem [tại đây](#)

<https://dev.to/ajeetraina/how-to-install-and-configure-nvm-on-mac-os-5fqi>

#60. Setup Dự Án Backend

Bước 1: Download/clone dự án thực hành [tại đây](#)

Bước 2: Cài đặt thư viện cần thiết
npm i

Bước 3: Cấu hình port (nếu muốn)
//json-server.json

Bước 4: Chạy dự án backend
npm run dev
<http://localhost:8000/>

#61. Xử Lý Bất Đồng Bộ (Asynchronous) là gì ?

1. Đồng Bộ (Synchronous) là gì?

Trong mô hình đồng bộ, các đoạn mã sẽ **thực hiện tuần tự**.

Mỗi dòng lệnh **phải hoàn thành xong** trước khi dòng tiếp theo được chạy.

Ví dụ:

```
console.log("1");  
console.log("2");  
console.log("3");
```

2. Bất Đồng Bộ (Asynchronous) là gì?

Trong mô hình bất đồng bộ, một số tác vụ có thể **chạy nền (non-blocking)**, cho phép chương trình **tiếp tục thực hiện các đoạn mã khác** mà không phải chờ đợi hoàn tất.

Đặc biệt quan trọng trong các thao tác như:

- **Chờ API phản hồi**
- Đọc/ghi file
- Lấy dữ liệu từ server
- Chờ sự kiện người dùng

Ví dụ:

```
console.log("1");  
setTimeout(() => {  
  console.log("2");  
}, 1000);  
console.log("3");
```

#62. Promise: Lời hứa từ tương lai

Tài liệu:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise#examples

1.Promise là gì?

Trong JavaScript, Promise là một đối tượng đại diện cho một giá trị sẽ có trong tương lai, thường là kết quả của một tác vụ bất đồng bộ như:

- Gọi API
- Đọc file
- Truy vấn cơ sở dữ liệu

Nói cách khác: Promise giống như một lời hứa:

"Tôi sẽ cung cấp kết quả sau – hoặc thành công, hoặc thất bại!"

3 trạng thái của Promise

Trạng thái	Mô tả
pending	Chưa có kết quả – đang xử lý
fulfilled	Thành công – trả về kết quả (resolve)
rejected	Thất bại – trả về lỗi (reject)

//todo: Sử dụng Promise: .then() và .catch()

#63. Gọi API với Fetch

1. API là gì?

API (Application Programming Interface) là giao diện giúp các ứng dụng giao tiếp với nhau. Trong web, bạn thường gọi API để lấy dữ liệu từ server – ví dụ: danh sách sản phẩm, thông tin người dùng, bài viết,...

Ví dụ: Gọi API từ <https://jsonplaceholder.typicode.com/users> để lấy danh sách người dùng giả lập.

Hiểu đơn giản: API là 1 url (được backend tạo ra), frontend sẽ gọi tới URL ấy để lấy dữ liệu (data)

2. Fetch là gì?

fetch() là một hàm tích hợp sẵn trong JavaScript, dùng để gửi yêu cầu HTTP đến một URL (API).

- Được giới thiệu từ ES6
- Trả về **Promise**
- Hỗ trợ đầy đủ các phương thức HTTP: GET, POST, PUT, DELETE,...

Cú pháp:

```
fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error("Có lỗi xảy ra:", error);
  });
```

url: Địa chỉ API

response.json() : Chuyển dữ liệu từ JSON sang Object

.then(): Xử lý kết quả

.catch(): Bắt lỗi nếu có vấn đề

#64. Xử lý lỗi với try/catch/finally

1. Lỗi (Error) là gì?

Lỗi là những vấn đề xảy ra trong quá trình thực thi chương trình, khiến chương trình bị dừng hoặc chạy sai.

Ví dụ:

- Truy cập biến không tồn tại
- Gọi API thất bại
- Chia cho 0 (trong một số ngôn ngữ)
- Cú pháp sai

2. Sử dụng try/catch

```
try {  
    // code có thể gây lỗi  
  
} catch (error) {  
    // xử lý lỗi ở đây  
  
} finally {  
    // luôn chạy sau cùng (nếu cần)  
}
```


#65. Callback và vấn đề Callback Hell

1.Callback là gì?

Trong JavaScript, một **callback** là **một hàm được truyền vào như một đối số** cho một hàm khác, và sẽ được **gọi lại** (callback) sau khi hàm kia thực hiện xong công việc của nó.

Callback thường được dùng trong các tác vụ bất đồng bộ (asynchronous) như:

- Đọc file
- Gọi API
- Chờ đợi sự kiện

Ví dụ:

```
function greet(name, callback) {  
  console.log("Hi " + name);  
  callback();  
}
```

```
function sayGoodbye() {  
  console.log("Goodbye!");  
}
```

```
greet("hoidanit", sayGoodbye);
```

2.Callback Hell là gì?

Callback Hell (địa ngục callback) là thuật ngữ mô tả **việc lồng quá nhiều callback bên trong nhau**, khiến code trở nên khó đọc, khó bảo trì..

- Mã lồng nhau nhiều tầng ("pyramid of doom" – hình kim tự tháp)
- Khó kiểm soát luồng dữ liệu
- Khó bắt lỗi
- Gây mệt mỏi cho lập trình viên khi bảo trì

Ví dụ về callback hell:

```
loginUser("hoidanit", function (user) {  
  getUserProfile(user.id, function (profile) {  
    getUserSettings(profile.id, function (settings) {  
      updateUI(settings, function () {  
        console.log("Hoàn thành!");  
      });  
    });  
  });  
});
```

#66. Async/Await: Cú pháp cho code sạch đẹp

1. Async/Await là gì?

async/await là cú pháp hiện đại nhất trong JavaScript để xử lý bất đồng bộ, được giới thiệu từ ES2017 (ES8).

Nó là một cách viết gọn gàng hơn cho Promise, giúp code đọc giống như code đồng bộ, dễ hiểu và dễ bảo trì hơn.

2. So sánh với Promise

Promise:

```
getUser()
  .then(user => getProfile(user))
  .then(profile => getSettings(profile))
  .then(settings => console.log(settings))
  .catch(err => console.error(err));
```

async/await

```
async function showSettings() {
  try {
    const user = await getUser();
    const profile = await getProfile(user);
    const settings = await getSettings(profile);
    console.log(settings);
  } catch (err) {
    console.error(err);
  }
}
```

#67. Bài Tập Lab 05

Yêu cầu: xây dựng table users với data từ backend

Bước 1: Bạn cần chạy backend
`npm run dev`

Kiểm tra backend đã hoạt động chưa:
<http://localhost:8000/>

Kiểm tra API đã hoạt động chưa:
<http://localhost:8000/users>

Bước 2: Thực hiện gọi API với fetch (lấy danh sách users)
`//sử dụng async await`

Bước 3: Sử dụng javascript render table

Render động dữ liệu của table html dựa vào javascript và data của api

Tham khảo template table html [tại đây](#)

Gợi ý cách sử dụng javascript để chèn động nội dung cho table:
`const tbody = document.querySelector("#myTable tbody");
tbody.innerHTML += `<tr><td>data 1</td><td> data 2</td></tr>`;`

#68. Chữa Bài Tập Lab 05
`//todo`

Chapter 8: Dự Án Thực Hành JavaScript

Luyện tập các kiến thức đã học về JavaScript thông qua các dự án thực hành

#69. Tổng quan về chapter 8

//todo

#70. Bài Tập Thực Hành 01

Yêu cầu: tạo ứng dụng todo list, kết hợp lưu trữ data với Local Storage

Bước 1: tạo mới todo

Tạo file **create.todo.html**, với nội dung sau:

form html để tạo mới todo, bao gồm:

- Input để nhập todo
- Button để submit

Khi nhấn nút submit, sẽ cần xử lý:

1. Lấy dữ liệu (tên todo) tại input
2. Lưu todo vào localStorage theo định dạng object { id, name}
Id của todo sẽ được random ngẫu nhiên (cần tham số này, để sau này xóa todo)

Ví dụ: data todo khi lưu: { id: 1, name: "learn javascript"}

LocalStorage sẽ lưu dưới định dạng array: localStorage.setItem(todo, data)

```
[  
  { id: 1, name: "learn javascript"},  
  { id: 2, name: "learn typescript"},  
  ...  
]
```

3. Lưu todo thành công, redirect về file **video70.html**

Bước 2: Hiển thị danh sách todo

Hiển thị danh sách todo dưới dạng table, data lấy từ localStorage, tương tự :

Id	Name	Action
1	Learn javascript	Button Xóa
2	Learn typescript	Button Xóa

Bước 3: Xử lý hành động xóa todo

Khi nhấn vào button xóa trong table, sẽ tiến hành xóa todo. Sử dụng JavaScript để:

1. Lấy id của todo cần xóa

Gợi ý:

//với html, gán [data attribute](#)

```
<td><button class="delete-btn" data-id="1">Xóa</button></td>
```

// với javascript gán sự kiện click cho tất cả nút có class "delete-btn"

```
document.querySelectorAll('.delete-btn').forEach(function(button) {  
  button.addEventListener('click', function() {  
    const id = button.getAttribute('data-id');  
  });  
});
```

2. Sử dụng hàm filter để xóa data localStorage
3. Xóa thành công, reload lại website để thấy kết quả

#71.1 Chữa Bài Tập Thực Hành 01 - Part 1

Mục tiêu: create todo

```
//random number  
function getRandomInt(min, max) {  
  min = Math.ceil(min);  
  max = Math.floor(max);  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}
```

#71.2 Chữa Bài Tập Thực Hành 01 - Part 2

Mục tiêu: display todo

```
//todo
```

#71.3 Chữa Bài Tập Thực Hành 01 - Part 3

Mục tiêu: delete a todo

```
//todo
```

#72. Bài Tập Thực Hành 02

Yêu cầu: xây dựng table blog với data từ backend

Bước 1: Bạn cần chạy backend
`npm run dev`

Kiểm tra backend đã hoạt động chưa:

<http://localhost:8000/>

Kiểm tra API đã hoạt động chưa:

<http://localhost:8000/blogs>

Bước 2: Render tables blogs

Thực hiện gọi API với fetch (lấy danh sách blogs)

//sử dụng async await

Id	Title	Author	Content	Action
1	...	...	...	Button Xóa
2	...	...	...	Button Xóa

Bước 3: Thêm mới blog

Tạo form thêm mới blog phía trên table (cùng page html), bao gồm

- 3 ô input: title, author, content

- 1 button submit

Khi nhấn nút submit thì:

Tạo mới blog bằng cách sử dụng fetch gọi API với method POST

<https://stackoverflow.com/a/29823632>

POST <http://localhost:8000/blogs>

body truyền lên object { title, author, content}

Sau khi tạo mới xong:

- Thêm row cuối của table với data vừa thêm mới

Gợi ý:

```
const tableBody = document.querySelector('#myTable tbody');
```

```
// Tạo phần tử dòng mới
```

```
const newRow = document.createElement('tr');
```

```
// Gán HTML cho dòng
```

```
newRow.innerHTML = `
```

```
  <td>Người mới ${currentId}</td>
```

```
  <td><button class="delete-btn" data-id="${currentId}">Xóa</button></td>
```

```
`;
```

```
// Thêm dòng vào cuối bảng
```

```
tableBody.appendChild(newRow);
```

Bước 4: Xóa blog

1. Lấy id của blog cần xóa

Gợi ý:

//với html, gán [data attribute](#)

```
<td><button class="delete-btn" data-id="1">Xóa</button></td>
```

// với javascript gán sự kiện click cho tất cả nút có class "delete-btn"

```
document.querySelectorAll('.delete-btn').forEach(function(button) {
```

```
  button.addEventListener('click', function() {
```

```
    const id = button.getAttribute('data-id');
```

```
  });
```

```
});
```

2. Gọi api để xóa blog

Xóa 1 blog (tham khảo sử dụng với fetch [tại đây](#))

DELETE <http://localhost:8000/blogs/:id>

3. Dùng javascript để xóa row

```
document.querySelectorAll('.delete-btn').forEach(function(button) {  
  button.addEventListener('click', function() {  
    const row = this.closest('tr');  
    row.remove();  
  
  });  
});
```

#73.1 Chữa Bài Tập Thực Hành 02 - Part 1

Mục tiêu: Hiển thị danh sách blog

//todo

#73.2 Chữa Bài Tập Thực Hành 02 - Part 2

Mục tiêu: Thêm mới blog

//todo

#73.3 Chữa Bài Tập Thực Hành 02 - Part 3

Mục tiêu: Xóa blog

//todo

#74. What's next với JavaScript

JavaScript có thể giúp bạn làm được website (frontend, backend), app mobile và nhiều ứng dụng hơn nữa...

Cách đơn giản, và phổ biến nhất của javascript (tương tự như cách khóa học đã hướng dẫn), sử dụng javascript sẽ giúp kiểm soát nội dung HTML/CSS một cách linh động.

Các định hướng phát triển tiếp theo để hiểu rõ và nắm vững JavaScript hơn, là **học chuyên sâu về các thư viện (library) và framework của javascript**.

Gợi ý các định hướng phát triển:

Khuyến khích chủ động nâng cấp/upgrade skill với ngôn ngữ TypeScript

Tham khảo khóa học TypeScript [tại đây](#)

Định hướng 1: Phát triển **frontend** của website với JavaScript

Bạn có thể học các thư viện, framework như React.js, Angular, Vue... Đây là những công cụ chuyên nghiệp, giúp bạn giảm thiểu thời gian phát triển giao diện website

Tham khảo khóa học React.js của Hỏi Dân IT [tại đây](#)

Định hướng 2: Phát triển **backend** của website với Node.js

Bạn có thể học các framework làm website phổ biến như Express, Nestjs...

Tham khảo khóa học Node.js Pro của Hỏi Dân IT [tại đây](#)

Định hướng 3: Phát triển **mobile app**

Bạn có thể tìm hiểu về React Native

Tham khảo khóa học React Native của Hỏi Dân IT [tại đây](#)

Định hướng 4: Các ứng dụng khác

JavaScript có thể giúp bạn làm desktop app, các ứng dụng IoT...

Tùy vào mục đích sử dụng, bạn cần tìm hiểu các thư viện/framework hỗ trợ lĩnh vực ấy

Chapter 9: Giới thiệu TypeScript

Hiểu TypeScript và lợi ích của typescript so với JavaScript

#75. Tổng quan về chapter 9

//todo

#76. TypeScript là gì ?

Tài liệu: <https://www.typescriptlang.org/>

1. TypeScript là gì?

TypeScript là một ngôn ngữ lập trình mã nguồn mở được phát triển bởi **Microsoft**, dựa trên JavaScript nhưng bổ sung thêm kiểu dữ liệu tĩnh (static types) và một số tính năng nâng cao dành cho lập trình viên.

Hiểu đơn giản:

TypeScript = JavaScript + Khai báo Types

Mối quan hệ giữa JavaScript và TypeScript:

TypeScript → (compile) → JavaScript → (chạy được)

2. Lợi ích của TypeScript

JavaScript rất linh hoạt nhưng cũng dễ xảy ra lỗi khi chương trình lớn dần:

- **Không kiểm tra kiểu dữ liệu** (dễ gây bug khó tìm).
- **Không có cảnh báo** khi sai cú pháp hoặc truyền sai tham số.
- Thiếu hỗ trợ mạnh mẽ cho lập trình hướng đối tượng (OOP).

TypeScript giải quyết các vấn đề đó bằng cách:

- ✓ Kiểm tra lỗi ngay khi lập trình (trước khi chạy).
- ✓ Gợi ý code thông minh hơn khi dùng với VSCode.
- ✓ Tăng tính ổn định và dễ bảo trì cho các dự án lớn.
- ✓ Hỗ trợ OOP đầy đủ (class, interface, inheritance...).

#77. Tôi bắt buộc phải biết TypeScript không ?

1. So sánh sự phổ biến của TypeScript

Package TypeScript: <https://www.npmjs.com/package/typescript>

Số lượng download của TypeScript:
<https://npmtrends.com/typescript>

Tôi bắt buộc phải biết TypeScript không ?

Các con số thống kê không biết nói dối. Nếu bạn là một lập trình viên JavaScript (frontend/backend) thì chắc chắn TypeScript là một công cụ không thể thiếu trên cuộc hành trình sự nghiệp của bạn.

Lượt download nhiều cho thấy các thư viện/framework đang dần chuyển đổi sang TypeScript như là một tiêu chuẩn.

2. Lợi ích khi nắm vững JavaScript

Tại sao mình lại hướng dẫn JavaScript trước khi học TypeScript ?

Suy cho cùng (at the end of the day), TypeScript sẽ là JavaScript (browser/server không hiểu typescript, chỉ hiểu JavaScript)

Đôi khi **bạn code typescript, bạn vẫn có thể viết code (logic) của JavaScript**, nó vẫn hoạt động bình thường (no bug, no error)

Nhưng, nếu bạn viết code dự án javascript, code thêm typescript vào, chắc chắn có lỗi.

Như vậy, việc biết JavaScript sẽ là phao cứu sinh, giúp bạn flexible tùy từng trường hợp gặp phải.

Và, nắm vững JavaScript, bạn cũng hiểu được phần nào cách hoạt động của TypeScript.

Hãy nhớ: **TypeScript = JavaScript + Types**

#78. Cài đặt môi trường Nodejs (TypeScript)

Tại video [#59. Cài đặt môi trường Nodejs \(JavaScript\)](#), đã hướng dẫn cài đặt môi trường Node.js

Nodejs không phải là thư viện (library), không phải framework của JavaScript.

Nodejs là môi trường để bạn thực thi code javascript, tại browser và server.

//todo

Link tải nodejs v22.13.0:

<https://nodejs.org/download/release/v22.13.0/>

Sau khi cài đặt xong, kiểm tra bằng cách gõ câu lệnh:

node -v

#79. Quy Trình Chạy Dự Án TypeScript

Bước 1: cài đặt môi trường nodejs (đã làm)

Nodejs ở đây là môi trường giúp thực thi code javascript/typescript

Bước 2: cài đặt thư viện hỗ trợ TypeScript

Trong quá trình coding, cần thư viện gợi ý, hỗ trợ dịch code TypeScript sang JavaScript

Cấu hình TypeScript (tsconfig.json) để biết quy tắc dịch code từ TypeScript sang JavaScript

Bước 3: dịch code TypeScript sang JavaScript

Khi chạy chương trình, máy tính sẽ cần dịch TypeScript sang JavaScript

Ví dụ: browser chỉ có khả năng chạy code javascript

#80. Cài đặt Typescript Compiler

<https://www.npmjs.com/package/typescript>

Yêu cầu bắt buộc: cài đặt môi trường Node.js

Kiểm tra bằng cách: **node -v**

Cài đặt typescript

npm install -g typescript@5.8.3

Kiểm tra: **tsc -v**

#81. Hello world với Typescript (Basic)

Bước 0: Tạo folder chứa source code

Lưu ý về tên dự án thực hành và đường dẫn (path) nơi lưu trữ code, không dùng ký tự đặc biệt và tiếng việt có dấu

Nên dùng: D://my-project/typescript-hoidanit

Không nên dùng: D//dự án/học typescript

Đặt tên dự án thực hành là: **typescript-hoidanit**

Bước 1: Cài đặt typescript (đã làm)

tsc -v

Bước 2: viết file typescript (.ts)

//todo

Bước 3: dịch file typescript sang javascript

tsc ten_file_typescript

Bước 4: chạy file javascript

node ten_file_javascript

#82. Setup Dự Án Thực Hành TypeScript (Node.js)

Bước 1: Tạo folder chứa source code (đã làm)

Lưu ý về tên dự án thực hành và đường dẫn (path) nơi lưu trữ code, không dùng ký tự đặc biệt và tiếng việt có dấu

Nên dùng: D://my-project/typescript-hoidanit

Không nên dùng: D://dự án/học typescript

Đặt tên dự án thực hành là: **typescript-hoidanit**

Bước 2: Tạo file **package.json**

Tạo ra file package.json – để quản lý package và script

npm init -y

Bước 3: Cài đặt TypeScript cục bộ (local).

Để đảm bảo môi trường đồng nhất giữa các máy dev

Không dùng -g, để tránh xung đột giữa các project.

npm install --save-dev --save-exact typescript@5.8.3

Bước 4: Cấu hình typescript

Tạo file cấu hình **tsconfig.json** mặc định: **npx tsc --init**

Gợi ý cấu hình basic cho dự án thực hành:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

Bước 5: Viết hello world

Tạo thư mục **src** và viết code trong đây

Dịch code với câu lệnh: **npx tsc**

Chạy ứng dụng

Bước 6: Tạo file **.gitignore**

//todo

#83. Chạy dự án với HTML

Bước 1: Tạo script để build dự án

Mục đích: dịch code từ typescript sang javascript

```
//package.json
"scripts": {
  "build": "tsc",
  "dev": "tsc --watch"
}
```

npm run build # Biên dịch toàn bộ

npm run dev # Biên dịch tự động khi có thay đổi (watch mode)

Bước 2: Tạo file html

Import code javascript từ thư mục dist

Bước 3: Run dự án

```
//todo
```

#84. Giải Thích Cấu Trúc Dự Án Thực Hành (Extra)

Ý nghĩa của các file/folder trong dự án thực hành:

package.json : thể hiện các thư viện cài đặt, cấu hình script để chạy dự án

package-lock.json: cấu hình chi tiết version, mối quan hệ... của các thư viện cài đặt

.gitignore: cấu hình các file/folder không đẩy lên git (remote repository)

tsconfig.json: cấu hình cách dịch code từ typescript sang javascript

node_modules: thư viện cài đặt trong dự án

src: thư mục **lưu code typescript**

dist: thư mục dịch code, **chứa code javascript**

index.html: file html chạy dự án

#85. Tài liệu về TypeScript (Docs)

Tham khảo:

<https://www.typescriptlang.org/docs/handbook/intro.html>

Chapter 10: Kiểu dữ liệu (Data Type) với TypeScript

Sử dụng kiểu dữ liệu (data type) trong TypeScript

#86. Tổng quan về chapter 10

//todo

#87. Vấn đề tồn đọng khi sử dụng JavaScript ?

1. Kiểu dữ liệu là gì?

Trong lập trình, **kiểu dữ liệu** là cách mà máy tính hiểu và lưu trữ giá trị.

Ví dụ: "Hello" là kiểu **chuỗi** (string), 123 là kiểu **số** (number), true là kiểu **boolean**.

JavaScript là ngôn ngữ kiểu động (dynamic typing).

Biến có thể thay đổi kiểu bất kỳ lúc nào mà không báo lỗi:

```
let x = 10;    // number
x = "hello";  // bây giờ là string
```

Điều này khiến chương trình **dễ bị lỗi khi chạy (runtime)**, vì sai kiểu mà không biết trước.

Ví dụ:

```
function greet(name) {
  console.log("Hello " + name.toUpperCase());
}
```

```
greet("hoidanit"); // OK
greet(123);        // ❌ lỗi ngay trong lúc viết
```

#88. String, Number và Boolean

JavaScript có 3 loại dữ liệu nguyên thủy (primitive) được dùng nhiều nhất: **string**, **number** và **boolean**

Mỗi loại type này, được sử dụng giống hệt (tên type/typeof) khi sử dụng với TypeScript (viết thường: **string** ; **number**; **boolean**)

Ví dụ:

```
const name: string = "hoidanit"
```

```
const age: number = 25;
```

```
const isHandsome: boolean = true;
```

Lưu ý: không sử dụng String, Number và Boolean để định nghĩa type (bad coding)

String, Number, Boolean là kiểu đối tượng (object), là wrapper (lớp bọc ngoài) cho kiểu dữ liệu nguyên thủy.

```
let myString: string = 'hello world';
```

```
// Khi bạn gọi .length hoặc .toUpperCase(),  
// JavaScript ngầm tạo ra một đối tượng new String(myString)  
// để truy cập các phương thức đó.  
console.log(myString.length); // 11  
console.log(myString.toUpperCase()); // "HELLO WORLD"
```


#89. Arrays

Cách khai báo mảng (array)

Cách 1: Dùng [] (phổ biến nhất)

```
let numbers: number[] = [1, 2, 3, 4];  
let names: string[] = ["hoidanit", "eric", "bla bla"];
```

Cách 2: Dùng Array<type> (phần này liên quan tới generic, sẽ được đề cập sau)

```
let numbers: Array<number> = [1, 2, 3, 4];  
let names: Array<string> = ["hoidanit", "eric", "bla bla"];
```

#90. any/unknown

1. any là gì?

any là một kiểu dữ liệu đặc biệt trong TypeScript. Khi một biến có kiểu any, biến đó có thể chứa bất kỳ giá trị nào — tương tự như JavaScript thuần.

Nói cách khác, any vô hiệu hóa kiểm tra kiểu của TypeScript.

ví dụ:

```
let value: any;  
value = 10;      // số  
value = "hello"; // chuỗi  
value = true;    // boolean  
value = [1, 2, 3]; // mảng  
value = { name: "hoidanit" }; // object
```

2. Khi nào nên dùng/không nên dùng any ?

Nên dùng any khi:

- Dữ liệu không rõ cấu trúc (API bên thứ ba)
- Code đang chuyển từ JavaScript sang TypeScript
- Khi bạn hiểu bạn đang làm gì (và không muốn khai báo type)

Không nên dùng any khi:

- Bạn muốn được gợi ý code, check type của typescript
- Bạn muốn code dễ mở rộng và bảo trì
- Teamsize/codebase lớn

3. noImplicitAny

//tsconfig.json

Cấu hình: "noImplicitAny": true

TypeScript sẽ báo lỗi nếu một biến, tham số, hoặc giá trị nào đó không được gán kiểu rõ ràng và TypeScript không thể suy ra kiểu — dẫn đến kiểu any được gán mặc định.

4. Sử dụng an toàn hơn với unknown

//todo

#91. Type Annotations/Type Inference

1. Type Annotations

Ví dụ:

```
const myName: string = "hoidanit";
```

:string gọi là type annotation (comment/giải thích)

Với typescript, **type annotation phải được khai báo sau tên của biến số** (không phải khai báo trước, ví dụ: `int x =`)

2. Type infer

Trong hầu hết các trường hợp, Typescript sẽ có cơ chế “infer” (tự động đoán type) phù hợp với ngữ cảnh coding.

Vì vậy, có thể bạn sẽ thấy coding typescript ít cần khai báo type hơn so với bạn nghĩ :)

```
// No type annotation needed -- 'myName' inferred as type 'string'  
let myName = "hoidanit";
```

#92. Functions

TypeScript cho phép quy định kiểu type cho input (tham số đầu vào) và output (kiểu trả về) của một function

1.Parameter Type Annotations

Khi định nghĩa một function, bạn có thể thêm type sau mỗi tham số khai báo. Type annotation được khai báo phía sau tên biến số

Ví dụ:

```
// Parameter type annotation
function greet(name: string) {
  console.log("Hello, " + name.toUpperCase() + "!!");
}
```

Khi định nghĩa type cho tham số đầu vào, typescript sẽ check type

Ví dụ:

```
greet(42); //Argument of type 'number' is not assignable to parameter of type 'string'.
```

2.Return Type Annotations

Bạn có thể định nghĩa type cho kiểu dữ liệu trả về. type annotations được khai báo phía sau danh sách tham số

Ví dụ:

```
function getFavoriteNumber(): number {
  return 96;
}
```

Đa phần (phần lớn thời gian) bạn sẽ không cần phải định nghĩa type cho kiểu trả về của function. TypeScript sẽ infer (tự động đoán) kiểu trả về dựa vào keyword **return**.

Nếu bạn dùng khai báo type cho function:

- muốn viết document cụ thể
- ngăn chặn việc thay đổi không chủ đích (vô tình thay đổi)
- sở thích cá nhân

#93. Union Types

1. Định nghĩa

Ngoài các datatype đã được định nghĩa sẵn, TypeScript cho phép bạn tự định nghĩa kiểu type bạn mong muốn.

Union Type là loại type được tạo ra từ 2 hoặc nhiều loại types khác.

Ví dụ:

```
function printId(id: number | string) {  
  console.log("Your ID is: " + id);  
}  
// OK  
printId(25);  
// OK  
printId("hoidanit");
```

2. Cách sử dụng

TypeScript chỉ cho phép xử lý toán tử, nếu nó "hợp lệ" với mọi kiểu type thuộc union types

Ví dụ:

```
function printId(id: number | string) {  
  console.log(id.toUpperCase());  
}
```

Property 'toUpperCase' does not exist on type 'string | number'.

Property 'toUpperCase' does not exist on type 'number'.
}

//giải pháp đề ra là "narrow type" (giới hạn type - phạm vi hẹp hơn)
typeof id === "string"

#94. Object Types

1. Định nghĩa type

Để định nghĩa object type, bạn cần định nghĩa thuộc tính (key) và kiểu type cho thuộc tính đấy.

Bạn có thể dùng **dấu phẩy** (,) hoặc **chấm phẩy** (;) để ngăn cách giữa các thuộc tính

Ví dụ:

```
// The parameter's type annotation is an object type
function printCoord(pt: { x: number; y: number }) {
  console.log("The coordinate's x value is " + pt.x);
  console.log("The coordinate's y value is " + pt.y);
}
printCoord({ x: 3, y: 7 });
```

2. Optinal Properties

Để định nghĩa 1 thuộc tính là optional, sử dụng dấu ? sau tên của thuộc tính.

Ví dụ:

```
function printName(obj: { first: string; last?: string }) {
  // ...
}
// Both OK
printName({ first: "hoidanit" });
printName({ first: "hoidanit", last: "with eric" });
```

#95. Type Aliases

Type Aliases là cách đặt tên cho bất cứ loại type nào bạn muốn.

Mục đích: tái sử dụng code thông qua tên của type

Ví dụ:

```
type Point = {  
  x: number;  
  y: number;  
};
```

// Exactly the same as the earlier example

```
function printCoord(pt: Point) {  
  console.log("The coordinate's x value is " + pt.x);  
  console.log("The coordinate's y value is " + pt.y);  
}
```

```
printCoord({ x: 50, y: 70 });
```

#96. Bài Tập Lab 06

Yêu cầu:

1. Tạo hai **đối tượng** học viên (student1, student2)

Mỗi học viên cần có các thông tin sau:

id: số nguyên (number)

name: chuỗi (string)

email: chuỗi (string)

isPremium: true nếu học viên đăng ký gói cao cấp, false nếu thường

contact: có thể là email (string) hoặc số điện thoại (number) → dùng union type

2. Tạo một **đối tượng** khóa học (course) có thông tin:

courseId: số nguyên (number)

title: tên khóa học (string)

price: giá gốc (number)

students: danh sách học viên đã đăng ký → mảng object chứa học viên

3. Viết hàm **registerStudentToCourse**(student, course)

Chức năng: Thêm học viên vào danh sách của khóa học.

4. Viết hàm **printCourseInfo**(course)

Chức năng:

- In ra tên khóa học và số học viên đã đăng ký.
- Nếu chưa có học viên → in "Chưa có học viên nào."
- Nếu có học viên → liệt kê tên từng học viên.

//hàm main

Thực hiện yêu cầu 1 và yêu cầu 2. Sau đấy thực thi 2 function **registerStudentToCourse** và **printCourseInfo** để in ra thông tin

#97. Chữa Bài Tập Lab 06

//todo

Chapter 11: Interface

Quản lý cấu trúc dữ liệu phức tạp với TypeScript

#98. Tổng quan về chapter 11

//todo

#99. null và undefined

Tương tự javascript, Typescript có hỗ trợ sử dụng null và undefined

undefined: khai báo biến số, và chưa gán giá trị cho nó

null: khởi tạo giá trị bằng null (ý nghĩa là không có giá trị /nothing/empty)

1.Cấu hình strictNullChecks

//tsconfig.json

strictNullChecks: off

không check null/undefined, đây có thể là nguyên nhân (tiềm tàng nguy cơ) dẫn tới bug nếu khối lượng codebase lớn.

strictNullChecks: on

Nếu cấu hình như trên, khi một giá trị là null/undefined, bạn cần phải kiểm tra giá trị trước khi sử dụng methods/properties của giá trị đấy

Ví dụ:

```
function doSomething(x: string | null) {  
  if (x === null) { // do nothing  
  } else {  
    console.log("Hello, " + x.toUpperCase());  
  }  
}
```

2.Non-null Assertion Operator (Postfix !)

Có thể dùng dấu chấm than (!) để nói với TypeScript một biến số khác null/undefined.

Tuy nhiên, không nên lạm dụng, và chỉ sử dụng khi bạn biết bạn đang làm gì, cũng như chắc chắn được giá trị của biến số khác null/undefined

#100. Enum/Literal types (Extra)

Tài liệu: <https://www.typescriptlang.org/docs/handbook/enums.html>

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#literal-types>

1.Sử dụng Literal types

Literal Types là kiểu dữ liệu mà giá trị của biến được giới hạn chỉ trong một số giá trị cụ thể

```
let direction: "left" | "right";
```

2.Enums

Enums (enumerations) trong TypeScript là kiểu dữ liệu đặc biệt dùng để định nghĩa một tập hợp các hằng số.

Chúng giúp mã nguồn trở nên dễ đọc, rõ ràng hơn và tránh lỗi sai do gán giá trị thủ công.

Enum dạng số (Numeric enums)

```
//todo
```

Enum dạng chuỗi (String Enums)

```
//todo
```

#101. tuple

Tuple là một kiểu dữ liệu mảng đặc biệt cho phép bạn xác định số lượng phần tử cụ thể, và kiểu dữ liệu của từng phần tử tại từng vị trí.

Hiểu đơn giản: **Tuple = mảng có cấu trúc cố định.**

Ví dụ:

```
let example: [string, number];
```

#102. Intersection Type

Intersection Type (kiểu giao nhau) là một tính năng của TypeScript cho phép bạn **kết hợp nhiều kiểu (type) lại thành một kiểu mới.**

Kiểu mới này sẽ kế thừa toàn bộ thuộc tính và phương thức của tất cả các kiểu được kết hợp.

Hiểu đơn giản: **A & B = một kiểu mới có tất cả thuộc tính của A và B.**

Ví dụ:

```
type A = { name: string };
```

```
type B = { age: number };
```

```
type Person = A & B;
```

#103. Interface là gì

Interface là một cách khác để định nghĩa datatype cho object

1. Interface là gì

Trong TypeScript, **interface** là một hợp đồng (contract) dùng để **mô tả cấu trúc của một đối tượng**. Nó **định nghĩa những thuộc tính và phương thức** mà một object cần phải có, nhưng **không chứa logic thực thi**.

Ví dụ:

```
interface Person {  
  name: string;  
  age: number;  
  greet(): void;  
}
```

2. Thao tác với interface

Interface với **optional property**

//todo

Interface với **readonly property (chỉ đọc)**

Ví dụ:

```
interface Car {  
  readonly id: number;  
  brand: string;  
}  
  
const car: Car = { id: 101, brand: "Toyota" };  
// car.id = 102; ❌ lỗi vì id là readonly
```

Interface mô tả Function

//todo

3. Interface mở rộng (extends)

Sử dụng **extends** để kế thừa giữa các interface

Ví dụ:

```
interface Animal {  
  name: string;  
}
```

```
interface Dog extends Animal {  
  breed: string;  
}
```

```
const myDog: Dog = {  
  name: "Buddy",  
  breed: "Golden Retriever",  
};
```

#104. Phân biệt Type Aliases và Interfaces

Tài Liệu:

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#differences-between-type-aliases-and-interfaces>

<https://stackoverflow.com/questions/37233735/interfaces-vs-types-in-typescript>

So sánh Type Aliases và Interfaces:

Tiêu chí	type (Type Alias)	interface (Giao diện)
Cách khai báo	type User = { name: string; age: number }	interface User { name: string; age: number }
Dùng cho kiểu object	✓	✓
Kết hợp kiểu (Union / Intersection)	✓ Hỗ trợ type Status = "success" "error" hoặc type A = B & C	✗ Không hỗ trợ
Hỗ trợ primitive / tuple	✓ Dùng cho mọi kiểu: string, number[], [number, string]...	✗ Chỉ dùng cho object
Kế thừa / mở rộng kiểu khác	✓ Dùng & để kết hợp nhiều kiểu (intersection)	✓ Dùng extends để kế thừa
Declaration Merging	✗ Không hỗ trợ – khai báo lại sẽ lỗi	✓ Hỗ trợ – có thể định nghĩa cùng tên nhiều lần và tự động gộp lại
Dễ mở rộng sau này	✗ Khó mở rộng (phải tạo lại type)	✓ Có thể mở rộng bằng cách extends hoặc merge nhiều lần
Nên dùng khi nào?	Khi cần union, tuple, primitive, intersection	Khi làm việc với object định hình rõ ràng và muốn mở rộng / kế thừa

Kinh nghiệm: Bạn có thể chọn chỉ dùng interface/type, cho tới khi bạn cần kiểu còn lại.

#105. Setup Dự Án TypeScript với HTML

Bước 1: Cài đặt Extension [Live Server](#)

//todo

Bước 2: Cấu hình tsconfig.json

//todo

Bước 3: Update file html với type module

//todo

#106. Type Assertions (Ép Kiểu Type)

Tài liệu:

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#type-assertions>

Type Assertions (ép kiểu trong TypeScript) là cú pháp cho phép bạn **nói với TypeScript rằng một giá trị có kiểu cụ thể**, dù TypeScript không tự suy luận ra điều đó.

Nói cách khác, bạn **ép buộc TypeScript tin rằng** biến của bạn có kiểu A thay vì kiểu mà nó suy luận.

1. Khi nào cần dùng?

- Khi TypeScript không đủ thông tin để suy đoán đúng kiểu.
- Khi bạn biết rõ kiểu dữ liệu hơn TypeScript, ví dụ: khi làm việc với API, DOM...
- Khi cần tắt cảnh báo tạm thời, nhưng bạn phải chắc chắn rằng kiểu bạn gán là đúng, nếu không sẽ gây lỗi khi runtime.

2. Cú pháp Type Assertion

dùng **as**

```
let value: unknown = "hello";  
let strLength = (value as string).length;
```

Khai báo type cho html, tham khảo [tại đây](#)

#107. Sử dụng TypeScript với fetch API (Promise)

Bước 1: Bạn cần chạy dự án backend
npm run dev

Kiểm tra backend đã hoạt động chưa:

<http://localhost:8000/>

Kiểm tra API đã hoạt động chưa:

<http://localhost:8000/users>

Bước 2: Thực hiện gọi API với fetch (lấy danh sách users)

//sử dụng async await

//ép kiểu dữ liệu để sử dụng data

#108. Bài Tập Lab 07

Yêu cầu: xây dựng table blog với data từ backend

Bước 1: Bạn cần chạy backend

npm run dev

Kiểm tra backend đã hoạt động chưa:

<http://localhost:8000/>

Kiểm tra API đã hoạt động chưa:

<http://localhost:8000/blogs>

Bước 2: Render tables blogs

Thực hiện gọi API với fetch (lấy danh sách blogs)

Id	Title	Author	Content
1	...	...	...
2	...	...	...

Render động dữ liệu của table html dựa vào javascript và data của api

Tham khảo template table html [tại đây](#)

```
const tbody = document.querySelector("#users tbody");
```

Dùng vòng lặp:

```
tbody.innerHTML += `  
<tr>  
  <td>${user.id}</td>  
  <td>${user.name}</td>  
  <td>${user.email}</td>  
</tr>`
```

#109. Chữa Bài Tập Lab 07

//todo

@hoidanit

Chapter 12: Class và OOP (Object Oriented Programming)

Sử dụng hàm và lập trình hướng đối tượng (OOP) với Typescript

#110. Tổng quan về chapter 12

//todo

#111. Lập Trình Hướng Đối Tượng Là Gì ? (OOP)

1.Khái niệm

Lập trình hướng đối tượng (OOP - Object-Oriented Programming) là một phương pháp lập trình dựa trên mô hình các "đối tượng" (object) trong thế giới thực.

Thay vì chỉ tập trung vào các hàm hay quy trình (như lập trình hướng thủ tục), OOP tổ chức chương trình thành các lớp (class) và đối tượng (object) tương tác với nhau.

Tác dụng: OOP giúp viết code rõ ràng, dễ quản lý, dễ mở rộng, và gần gũi với cách con người suy nghĩ trong cuộc sống thực.

Ví dụ: shopee

2.Đặc điểm

- **Đóng gói (Encapsulation)** – gom dữ liệu và các hành vi liên quan vào cùng một đối tượng.
- **Kế thừa (Inheritance)** – cho phép một lớp kế thừa thuộc tính/phương thức từ lớp khác.
- **Đa hình (Polymorphism)** – cùng một phương thức nhưng thực hiện khác nhau tùy đối tượng.
- **Trừu tượng (Abstraction)** – che giấu chi tiết phức tạp, chỉ hiển thị phần quan trọng.

#112. Class và Object trong TypeScript

Để mô phỏng "real world", chúng ta cần các "đối tượng" (target)

Đối với 1 sinh vật/đồ vật trong thế giới thực, đều có điều miêu tả bởi 2 thứ:

- Đặc điểm (thuộc tính)
- Hành vi đặc trưng của nó

Ví dụ:

Miêu tả con sư tử: (sinh vật)

- **Đặc điểm** (thuộc tính):

- + Có lông
- + thuộc họ mèo

- **Hành vi:**

- + đi săn mồi

Miêu tả 1 bạn sinh viên: (sinh vật)

- **Đặc điểm:**

- + có mã số sinh viên
- + có địa chỉ sinh sống
- + có tên lớp học

- **Hành vi:**

- + ôn tủ cho qua môn
- + ngủ nướng qua ngày
- + cuối tuần đi chơi

Miêu tả 1 cái máy tính (đồ vật)

- **Đặc điểm:**

- + có dung lượng ổ cứng (SSD/HDD)
- + có dung lượng bộ nhớ (RAM)

- **Hành vi:**

- + có thể on/off/restart

=> Sử dụng "đối tượng" để mô hình hóa.

Tuy nhiên, chúng ta không "miêu chỉ chi tiết", chỉ miêu tả "đối tượng tổng quát" (đây gọi là **tính trừu tượng hóa/abstraction**)

Ví dụ, thay vì miêu tả cụ thể sinh viên A, B, C, D ... chúng ta gọi chung là sinh viên:

Đối tượng tổng quát được gọi là **Class** (Lớp), còn mỗi sinh viên cụ thể gọi là **Object** (đối tượng).

Ví dụ: **class** SinhVien gồm:

- **Thuộc tính** (đặc điểm)

+ Tên

+ mã số sinh viên

- **Hành vi:**

+ ăn/ngủ/nghỉ/ôn thi

=> Để có tính tái sử dụng code cao, **OOP chỉ "định nghĩa" lớp tổng quát.**
còn nếu muốn có chi tiết => chúng ta tự tạo ra

Ví dụ, để tạo ra sinh viên A thì:

+ gán Tên = A

+ gán mã số sinh viên = 1234

Tương tự, tạo ra sinh viên B thì:

+ gán Tên = B

+ gán mã số sinh = 6789

...

Khái niệm:

- **Class** chính là "đối tượng tổng quát" của real-word được mô phỏng (class là bản thiết kế, mô tả khái quát)

- **Object** là các thực thể chi tiết của class đấy (instance)

#113. Thuộc tính và Phương thức (Fields & Methods)

1. Thuộc tính: (Fields)

<https://www.typescriptlang.org/docs/handbook/2/classes.html#fields>

Fields là các biến được khai báo bên trong class, đại diện cho dữ liệu hoặc trạng thái của đối tượng.

Ví dụ khai báo:

```
class Person {  
  name: string;  
  age: number;  
}
```

2. Phương thức (Methods)

<https://www.typescriptlang.org/docs/handbook/2/classes.html#methods>

Methods là các hàm được khai báo trong class, thể hiện **hành vi** của đối tượng.

Ví dụ:

```
class Person {  
  name: string;  
  
  greet(): void {  
    console.log(`Hello, I'm ${this.name}`);  
  }  
}
```

//todo: giải thích về keyword **this**

#114. Constructor – Hàm tạo

Tài liệu: <https://www.typescriptlang.org/docs/handbook/2/classes.html#constructors>
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/constructor>

1. Khái niệm

constructor là một phương thức đặc biệt trong class, được tự động gọi khi một đối tượng mới được tạo ra từ class đó.

Mục đích chính:

- Khởi tạo giá trị cho thuộc tính (fields)
- Thiết lập trạng thái ban đầu cho đối tượng

Cú pháp:

```
class ClassName {  
  constructor(parameters) {  
    // Khởi tạo thuộc tính  
  }  
}
```

Lưu ý: Nếu bạn không khai báo constructor, TypeScript sẽ tạo một constructor mặc định tương đương với:

```
constructor() {}
```

2. Constructor Overload

```
// ✅ Khai báo các cách khởi tạo  
constructor();  
constructor(username: string, age: number);  
  
// ✅ Phần thân thực sự duy nhất  
constructor(username?: string, age?: number) {  
  this.username = username ?? "Guest";  
  this.age = age ?? 0;  
}
```


#115. Access Modifiers – Giới hạn truy cập

Access Modifiers (giới hạn truy cập) là từ khóa dùng để **kiểm soát phạm vi truy cập** đến thuộc tính (fields) hoặc phương thức (methods) bên trong class.

Tác dụng:

- Ẩn thông tin nội bộ
- Bảo vệ dữ liệu
- Kiểm soát cách đối tượng bị truy cập hoặc chỉnh sửa

Các loại Access Modifiers trong TypeScript:

Modifier	Mô tả	Có thể truy cập từ đâu?
public	Mặc định. Ai cũng có thể truy cập	Mọi nơi
private	Chỉ truy cập được bên trong class	Bên trong class đó
protected	Truy cập được trong class và class con	Class đó và class kế thừa

#116. Tính đóng gói (getter/setter) (Encapsulation)

Tài liệu:

<https://www.typescriptlang.org/docs/handbook/2/classes.html#getters--setters>

Encapsulation (Tính đóng gói) là **nguyên lý cốt lõi của lập trình hướng đối tượng**, giúp **gộp dữ liệu (fields)** và **hành vi (methods)** vào một đối tượng duy nhất,

đồng thời **giấu đi chi tiết nội bộ**, chỉ cho phép truy cập thông qua giao diện (interface) được thiết kế rõ ràng.

//sử dụng getter/setter

#117. Kế thừa trong TypeScript (Inheritance)

Tài liệu:

<https://www.typescriptlang.org/docs/handbook/2/classes.html#extends-clauses>

1. Kế thừa là gì?

Kế thừa (Inheritance) là một trong những tính chất cốt lõi của lập trình hướng đối tượng.

Nó cho phép **một lớp con (child class) kế thừa các thuộc tính (fields) và phương thức (methods) từ một lớp cha (parent class)**, nhờ đó giúp **tái sử dụng mã code**, mở rộng chức năng mà không cần viết lại toàn bộ.

TypeScript sử dụng từ khóa **extends** để thể hiện quan hệ kế thừa

Ví dụ:

```
class Dog extends Animal { }
```

2. Các tính chất khác

- Làm sao để xác định được đâu là class con/class cha (is a - một loại)
Typescript chỉ hỗ trợ đơn kế thừa với extends
- Overriding Methods : ghi đè method
- Sử dụng super

#118. Interface và Implements

Tài liệu:

<https://www.typescriptlang.org/docs/handbook/2/classes.html#implements-clauses>

1.Interface và Class – Implements

Một class có thể **implements** (triển khai) một hoặc nhiều interface bằng từ khóa **implements**.

Khi đó, class bắt buộc phải cài đặt đầy đủ các thành phần của interface.

TypeScript hỗ trợ implements nhiều interface cùng lúc, giúp mô phỏng kế thừa đa hình (multiple inheritance) một cách an toàn.

Câu hỏi: khi nào dùng interface ?

//todo

#119. Tính Đa Hình (Polymorphism)

Đa hình (Polymorphism) là một đặc tính cốt lõi trong lập trình hướng đối tượng (OOP),

cho phép cùng một phương thức hoặc hành động nhưng có thể thực hiện theo nhiều cách khác nhau tùy theo đối tượng cụ thể.

Ví dụ:

```
// Sử dụng đa hình
const animals: Animal[] = [
  new Dog("Buddy"),
  new Cat("Whiskers"),
  new Animal("Creature"),
];

for (const a of animals) {
  a.speak(); // Tự động gọi đúng phương thức dựa trên kiểu thực thể
}
```

#120. Tính Trừu Tượng - Abstract Class (Abstraction)

1. Abstract Class là gì?

abstract class (lớp trừu tượng) là một lớp không thể được khởi tạo trực tiếp, chỉ được kế thừa bởi các lớp con.

Nó có thể chứa cả phương thức đã được cài đặt và phương thức trừu tượng (chưa có phần thân, bắt buộc lớp con phải triển khai).

Mục đích sử dụng:

- Dùng để định nghĩa khung xương chung cho các lớp con.
- Đảm bảo các lớp con phải triển khai một số phương thức quan trọng.
- Hữu ích trong các hệ thống có nhiều lớp con chia sẻ logic cơ bản nhưng cần override hành vi cụ thể.

2. Cú pháp

```
abstract class Animal {  
  constructor(public name: string) {}  
  
  abstract makeSound(): void; // phương thức trừu tượng  
  
  move(): void {  
    console.log(`${this.name} is moving.`);  
  }  
}
```

#121. Bài Tập Lab 08

1. Class User (cha):

Thuộc tính:

name: string (**public**)

email: string (**private**)

role: "student" | "teacher" (**protected**)

Sử dụng Constructor: Khởi tạo thông tin user với constructor. Nếu không truyền role thì mặc định là "student".

Phương thức:

get email() → trả về email

set email(email: string) → cập nhật email nếu hợp lệ

getInfo() → trả về thông tin người dùng

2. Kế thừa (Inheritance):

Class **Teacher**:

Thuộc tính: **courses:** string[]

Phương thức: **addCourse**(courseName: string): void

printUserInfo : [Teacher] **Eric** - Courses Taught: **React**

Class **Student**:

Thuộc tính: **enrolledCourses:** string[]

Phương thức: **enroll**(courseName: string): void

printUserInfo : [Student] **Hoidanit** - Enrolled Courses: **TypeScript Pro**

3. Tính Đa Hình (Polymorphism):

Tạo hàm **printUserInfo**(user: User)

→ Dùng tính đa hình để hiển thị thông tin khác nhau tùy theo loại user (Teacher hoặc Student).

Output mong muốn:

```
const hoidanit = new Student("Hoidanit", "hoidanit@email.com");  
hoidanit.enroll("TypeScript Pro");
```

```
const eric = new Teacher("Eric", "eric@email.com", "teacher");  
eric.addCourse("React");
```

```
printUserInfo(hoidanit);
```

```
// 👉 Output:
```

```
// [Student] Hoidanit - Enrolled Courses: TypeScript Pro
```

```
printUserInfo(eric);
```

```
// 👉 Output:
```

```
// [Teacher] Eric - Courses Taught: React
```

#122. Chữa Bài Tập Lab 08

```
//todo
```


Chapter 13: Generic

Tái sử dụng code & tăng khả năng mở rộng code TypeScript với Generic

#123. Tổng quan về chapter 13

//todo

#124. Giới thiệu về Generic

Ví dụ về generic, tham khảo [tại đây](#)

1. Generic là gì?

Generic là một cơ chế giúp bạn định nghĩa **code có thể tái sử dụng với nhiều kiểu dữ liệu**, mà không cần đánh đổi sự rõ ràng hay kiểm soát về kiểu.

Bạn có thể hình dung Generic như một “biến kiểu” — cho phép bạn viết linh hoạt nhưng vẫn an toàn.

Ví dụ: Thay vì viết hai hàm riêng cho mảng số và mảng chuỗi, bạn chỉ cần viết một hàm duy nhất sử dụng Generic, và TypeScript sẽ tự hiểu kiểu dữ liệu bạn đang làm việc.

2. Vì sao nên học Generic?

Generic là công cụ không thể thiếu trong các dự án thực tế:

- Viết code tái sử dụng mà không cần sao chép nhiều lần cho từng kiểu dữ liệu.
- Giữ được type safety – tránh lỗi khi truyền sai kiểu.
- Xây dựng các hàm, class, interface linh hoạt và mạnh mẽ.
- Cực kỳ hữu ích khi làm việc với mảng, API, form, hoặc viết thư viện dùng chung.

#125. Generic Functions (Hàm Generic)

1. Khái niệm

Hàm Generic là hàm có thể hoạt động với **nhều kiểu dữ liệu khác nhau** mà vẫn đảm bảo **an toàn kiểu (type safety)**.

Thay vì cố định kiểu dữ liệu đầu vào hoặc đầu ra, bạn sử dụng tham số kiểu (generic type parameter) để viết hàm **linh hoạt và tái sử dụng cao**.

Vấn đề đặt ra:

```
function identityNumber(value: number): number {  
    return value;  
}
```

```
function identityString(value: string): string {  
    return value;  
}
```

Nhược điểm: Mỗi kiểu dữ liệu phải viết một hàm riêng $\Rightarrow$ rườm rà, không linh hoạt.

2. Hàm Generic

Cú pháp:

```
function identity<T>(value: T): T {  
    return value;  
}
```

Trong đó:

T là generic type parameter — một “placeholder” cho kiểu dữ liệu thực tế.

value: T nghĩa là tham số value sẽ nhận kiểu bất kỳ mà người dùng chỉ định.

: T phía sau nghĩa là hàm sẽ trả về kiểu dữ liệu tương tự.

//todo: lấy phần tử đầu tiên của array

#126. Sử dụng nhiều Type Variable

1. Type Variable là gì?

Type Variable (hay còn gọi là generic type parameter) là một biến đại diện cho kiểu dữ liệu chưa xác định — bạn sẽ gán kiểu cụ thể khi sử dụng hàm, interface, hoặc class.

Thông thường, bạn thấy một biến kiểu T, nhưng TypeScript cũng cho phép bạn dùng nhiều biến kiểu cùng lúc: T, U, V...

Ví dụ:

```
function merge<T, U>(a: T, b: U): [T, U] {  
  return [a, b];  
}
```

Trong đó:

- T đại diện cho kiểu của tham số a.
- U đại diện cho kiểu của tham số b.
- Kết quả là một tuple [T, U].

#127. Generic Types - Kiểu dữ liệu tổng quát

1. Generic Types là gì?

Generic Types (Kiểu dữ liệu tổng quát) là một tính năng trong TypeScript cho phép bạn **tạo ra kiểu dữ liệu linh hoạt và tái sử dụng**, bằng cách sử dụng **tham số kiểu (type parameter)** thay vì gán cố định một kiểu cụ thể.

Ví dụ:

```
type StringArray = string[];  
type NumberArray = number[];
```

Có thể tổng quát:

```
type MyArray<T> = T[];
```

```
const names: MyArray<string> = ["hoidanit", "eric"];  
const ages: MyArray<number> = [20, 30];
```

```
//todo: sử dụng với type:  
type Wrapper<T> = {  
  value: T;  
};
```

#128. Generic Classes (Lớp Generic)

1. Generic Class là gì?

Generic Class là lớp (class) trong TypeScript được định nghĩa với tham số kiểu (type parameter), cho phép bạn tạo các đối tượng với nhiều kiểu dữ liệu khác nhau mà không cần viết lại logic.

Vấn đề đặt ra:

```
class NumberBox {  
  content: number;  
  
  constructor(value: number) {  
    this.content = value;  
  }  
  
  getContent(): number {  
    return this.content;  
  }  
}
```

#129. Generic Interfaces (Interface Generic)

Yêu cầu: chạy dự án backend

1. Interface Generic là gì?

Generic Interface là một interface có sử dụng tham số kiểu (type parameter) để giúp cấu trúc dữ liệu linh hoạt hơn – có thể đại diện cho nhiều kiểu dữ liệu khác nhau mà vẫn giữ sự an toàn về kiểu.

Vấn đề đặt ra:

```
interface StringResult {  
  data: string;  
}
```

```
const result: StringResult = { data: "Hello" }; // OK  
// Nhưng nếu muốn chứa number thì phải tạo interface khác
```

#130. Generic Constraints (Ràng buộc Generic)

1. Vấn đề đặt ra

Mặc định, **Generic nhận mọi kiểu**:

```
function printValue<T>(value: T): void {  
  console.log(value);  
}
```

TypeScript cho phép T nhận bất kỳ kiểu dữ liệu nào: number, string, object, boolean, array, v.v.

```
function logLength<T extends { length: number }>(value: T): void {  
  console.log(value.length);  
}
```

Cú pháp:

```
function exampleFunction<T extends ConstraintType>(arg: T): T {  
  return arg;  
}
```

2. Các ràng buộc hay gặp

//todo

- Ràng buộc với Interface
- Ràng buộc kiểu với Class
- Ràng buộc với keyof (thuộc tính của object)

#131. Default Generic Type

1. Default Generic Type là gì?

Default Generic Type cho phép bạn gán một kiểu mặc định cho tham số kiểu (type parameter) trong generic.

Khi người dùng không chỉ định kiểu cụ thể, TypeScript sẽ tự động dùng kiểu mặc định đó.

Cú pháp:

```
function func<T = DefaultType>(param: T): T {  
  return param;  
}
```

Điều này giúp generic trở nên dễ dùng hơn, đặc biệt với người dùng mới — họ không cần truyền quá nhiều tham số nếu đã có giá trị mặc định hợp lý.

Ví dụ:

```
function identity<T = string>(value: T): T {  
  return value;  
}
```

T = string: Nếu người dùng không truyền kiểu T, thì mặc định T sẽ là string.

#132. Các Utility Type có sẵn (Built-in)

Tài liệu: <https://www.typescriptlang.org/docs/handbook/utility-types.html>

Utility Types là các kiểu dữ liệu được **TypeScript cung cấp sẵn**, giúp bạn biến đổi, kết hợp thông tin từ các kiểu dữ liệu khác.

Chúng thường dùng để làm việc với object type theo cách linh hoạt hơn mà không cần viết lại kiểu từ đầu.

Các Utility Types phổ biến:

Tên Utility	Mục đích chính
Partial<T>	Biến tất cả thuộc tính của T thành tùy chọn (optional)
Required<T>	Biến tất cả thuộc tính thành bắt buộc
Readonly<T>	Làm cho toàn bộ object trở thành chỉ đọc
Pick<T, K>	Chọn ra một số thuộc tính từ T
Omit<T, K>	Loại bỏ một số thuộc tính khỏi T
Record<K, T>	Tạo object có các key thuộc K, giá trị kiểu T
Exclude<T, U>	Loại bỏ các kiểu U khỏi T
Extract<T, U>	Giữ lại các kiểu U nằm trong T
ReturnType<T>	Lấy kiểu trả về của hàm
Parameters<T>	Lấy kiểu của các tham số trong hàm

Chapter 14: Module

Tối ưu hóa việc tái sử dụng, import, export code với module.

#133. Module là gì ?

Tài liệu:

<https://www.typescriptlang.org/docs/handbook/modules/theory.html>

1. Khái niệm

Trong TypeScript (và JavaScript hiện đại), **module là một đơn vị mã (code unit) độc lập**, có thể xuất (export) và nhập (import) dữ liệu (biến, hàm, class, interface...) từ file khác.

Mỗi file .ts được coi là một module nếu nó có sử dụng **export** hoặc **import**.

2. Tại sao cần dùng Module?

- **Tổ chức code tốt hơn**: chia nhỏ logic thành các phần riêng biệt.
- **Tái sử dụng**: các hàm, class, interface có thể tái sử dụng ở nhiều nơi.
- **Giải xung đột tên** (namespace conflict): mỗi module có không gian tên riêng.
- **Dễ bảo trì**: code rõ ràng, dễ kiểm soát sự phụ thuộc giữa các phần.

#134. Tạo module trong TypeScript

1. Khi nào một file được coi là module?

Một file **.ts** được coi là **module** khi trong file đó có sử dụng:

export (xuất ra ngoài)

Hoặc **import** (nhập từ file khác)

Nếu không có **export** hoặc **import**, file đó chỉ là script thông thường, không có phạm vi module riêng biệt.

2. Cách tạo module

Dùng **export** để cho phép sử dụng từ file khác.

Khi **import**, dùng đúng tên file và đúng tên biến/hàm đã **export**.

//todo

#135. Module Scope

Module Scope là phạm vi riêng của từng file module.

Biến hoặc hàm không export sẽ chỉ tồn tại trong file hiện tại.

Đây là cơ chế mặc định của TypeScript (và JavaScript ES6+) giúp code an toàn, rõ ràng, và dễ bảo trì.

Tất cả các file có **import** hoặc **export** đều tự động được xem là module – và do đó có module scope riêng biệt.

#136. Named export vs Default export

Khi sử dụng module trong TypeScript (hoặc JavaScript hiện đại), bạn có thể xuất dữ liệu ra ngoài module theo hai cách chính:

Named Export – xuất nhiều phần tử có tên riêng biệt.

Default Export – chỉ định một phần tử làm "giá trị mặc định" của module.

1. Named Export – Xuất có tên

// file: math.ts

export const PI = 3.14;

export function add(a: number, b: number): number {
 return a + b;
}

Import tương ứng: import { PI, add, subtract } from './math';

Đặc điểm:

- Có thể export nhiều biến, hàm, class cùng lúc.
- Phải dùng đúng tên khi import.
- Có thể đổi tên khi import bằng **as**:
import { add as addition } from './math';

2. Default Export – Xuất mặc định

// file: logger.ts

export default function log(message: string) {
 console.log(`[LOG] \${message}`);
}

// file: app.ts

import log from './logger'; log('Hello TypeScript!');

Đặc điểm:

- Mỗi file chỉ được **1 giá trị default export**
- Khi import không cần { }, tên biến tự đặt tùy ý
Thích hợp khi module chỉ có 1 chức năng chính (ví dụ: 1 class, 1 hàm chính...)

#137. Import toàn bộ module

Cú pháp import * as

Cú pháp này cho phép bạn import toàn bộ nội dung được export từ một module vào một đối tượng duy nhất, thường được dùng như một namespace.

```
import * as TênModule from './đường-dẫn';
```

#138. So sánh CommonJS và ESM module

1. Tổng quan

Tiêu chí	CommonJS (CJS)	ESModule (ESM)
Nguồn gốc	Được Node.js phát triển từ đầu (2009)	Chuẩn ECMAScript chính thức (ES6+) (2015)
Cú pháp	require() và module.exports	import và export
Sử dụng với TypeScript	mặc định trong Node.js	Cần cấu hình "module": "ESNext" và "type": "module"

2. Khi nào dùng cái nào

Trường hợp	Nên dùng
Dự án Node.js truyền thống (backend)	CommonJS
Viết script CLI đơn giản	CommonJS
Dự án frontend với bundler	ESModule
Viết thư viện cho frontend hiện đại	ESModule
Dự án Node.js mới cần top-level await	ESModule
Tree Shaking (loại bỏ code thừa)	ESModule

#139. Cấu hình module trong tsconfig.json

Trường "**module**" trong **tsconfig.json** quy định cách TypeScript biên dịch module sang JavaScript tương ứng.

Cụ thể, nó xác định cú pháp đầu ra:

- Sử dụng require (CommonJS)
- Hay import/export (ESModule)

Các giá trị "module" phổ biến:

Giá trị	Mô tả	Môi trường sử dụng
CommonJS	Dịch sang require , module.exports	Node.js truyền thống
ES6 hoặc ES2015	Dịch sang import , export	Trình duyệt hiện đại, hoặc bundler
ESNext	Giữ nguyên cú pháp module ES mới nhất	Vite, Rollup, Webpack (ESM)
NodeNext	Hỗ trợ module .mjs, cấu hình "type": "module"	Node.js 16+ dùng ESM module
MD/ System /UMD	Dùng cho các hệ thống module cũ như RequireJS, SystemJS	❌ Hiếm dùng hiện nay

#140. Tổ chức dự án theo module

Các cách tổ chức module phổ biến

1.Theo chức năng (Functional-based)

Ví dụ app quản lý sản phẩm:

src/

```
|— products/
|   |— product.model.ts
|   |— product.service.ts
|   |— product.controller.ts
|— users/
|   |— user.model.ts
|   |— user.service.ts
|   |— user.controller.ts
|— config/
|   |— index.ts
|— app.ts
```

2.Theo tầng kiến trúc (Layer-based)

src/

```
|— models/
|   |— user.model.ts
|   |— product.model.ts
|— services/
|   |— user.service.ts
|   |— product.service.ts
|— controllers/
|   |— user.controller.ts
|   |— product.controller.ts
|— utils/
|   |— logger.ts
|— config/
|   |— env.ts
|— app.ts
```


#141. Keyword Protected (Extra)

protected là một access modifier trong lập trình hướng đối tượng.

Nó cho phép thành viên (thuộc tính hoặc phương thức) **chỉ được truy cập bên trong class đó và các class kế thừa**, nhưng không thể truy cập từ bên ngoài.

So sánh nhanh với các từ khóa khác:

Modifier	Trong class	Trong class kế thừa	Từ bên ngoài
public	✓ Có thể	✓ Có thể	✓ Có thể
protected	✓ Có thể	✓ Có thể	✗ Không thể
private	✓ Có thể	✗ Không thể	✗ Không thể

Chapter 15: Dự Án Thực Hành TypeScript (Project 03)

Ôn tập và thực hành các kiến thức typescript thông qua các projects

#142. Hướng Dẫn Tạo Dự Án Thực Hành 03

Yêu cầu với dự án thực hành:

- Dùng typescript
- Viết code theo module (import/export) - nếu cần thiết
- Chạy dự án với liveserver
Lưu ý tsconfig.json, đồng thời cấu hình html type module

//todo: viết hello world

#143. Tích Hợp Bootstrap

1. Bootstrap là gì ?

- Đây là framework **front-end** mã nguồn mở do Twitter phát triển.
- Giúp bạn **xây dựng giao diện web đẹp**, chuẩn responsive, dễ dùng **mà không cần viết CSS từ đầu**.

2. Tích hợp Bootstrap

Lưu ý: version Bootstrap mình sử dụng trong khóa học là version 5.x

Tham khảo:

<https://getbootstrap.com/docs/5.2/getting-started/introduction/#quick-start>

Tích hợp CSS (phần head của html):

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-rbsA2VBKQhggwzxH7pPCaAqO46MgnOM80zW1RWuH61DGLwZJEdK2Kadq2F9CUG65" crossorigin="anonymous">
```

Tích hợp JS (animation - trước khi đóng thẻ body)

```
<script src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.11.6/dist/umd/popper.min.js" integrity="sha384-oBqDVmMz9ATKxleP9tiCxS/Z9fNfEXiDAYTujMAeBAsjFuCZSmKbSSUnQlMh/jp3" crossorigin="anonymous"></script>
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/js/bootstrap.min.js" integrity="sha384-cuYeSxntonz0PPNlHhBs68uyIAVpIlOZZ5JqeqvYYIcEL727kskC66kF92t6Xl2V" crossorigin="anonymous"></script>
```

//todo: kiểm tra có tích hợp thành công hay không ?

#144. Hướng Dẫn Sử Dụng Component Bootstrap

Các components hay dùng (được dùng tại các video thực hành tiếp theo):

Button: <https://getbootstrap.com/docs/5.2/components/buttons/>

Input: <https://getbootstrap.com/docs/5.2/forms/overview/>

Table: <https://getbootstrap.com/docs/5.2/content/tables/>

#145. Hướng Dẫn Sử Dụng Modal

1.Modal là gì

Tham khảo:

<https://getbootstrap.com/docs/5.2/components/modal/>

2.Tích hợp Modal

Bước 1: đảm bảo rằng bạn đã cài đặt/tích hợp Bootstrap, đã làm tại video [#143](#)

Bước 2: tạo html của modal

<https://getbootstrap.com/docs/5.2/components/modal/#live-demo>

```
<!-- Button trigger modal -->
<button type="button" class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#exampleModal">
  Launch demo modal
</button>

<!-- Modal -->
<div class="modal fade" id="exampleModal" tabindex="-1" aria-labelledby="exampleModalLabel"
aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h1 class="modal-title fs-5" id="exampleModalLabel">Modal title</h1>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
      </div>
      <div class="modal-body">
        ...
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
        <button type="button" class="btn btn-primary">Save changes</button>
      </div>
    </div>
  </div>
</div>
```

3. Các thao tác thường gặp

- đóng/mở modal
- lấy giá trị input trên modal
- xử lý sự kiện submit

Có 2 cách chính hay dùng để tương tác với Modal:

Cách 1: Sử dụng data attributes

<https://getbootstrap.com/docs/5.2/components/modal/#via-data-attributes>

Sử dụng kết hợp với event:

<https://getbootstrap.com/docs/5.2/components/modal/#events>

Cách 2: Sử dụng javascript

<https://getbootstrap.com/docs/5.2/components/modal/#via-javascript>

#146. Demo Dự án Thực Hành 03

Yêu cầu: tạo ứng dụng todo list, kết hợp lưu trữ data với Local Storage **sử dụng TypeScript**

Nguyên tắc:

- Bạn xem video yêu cầu (gợi ý cách làm) để tự thực hành tính năng
- Xem đáp án (gợi ý cách làm) tại video tiếp theo

#147. Bài Tập Thực Hành 03 - Part 1

Lưu ý: với file html, cần nhúng js/css của Bootstrap, tương tự video [#143](#)

Tính năng 1: tạo mới todo

Tạo file **home.html**, trong file này, có button **"Add new Todo"**

Khi nhấn vào button trên, sẽ cần mở [Modal](#) với nội dung sau:

form để tạo mới todo, bao gồm:

- Input để nhập todo
- Button để submit

Khi nhấn nút submit, sẽ cần xử lý:

- Lấy dữ liệu (tên todo) tại input
- **Lưu todo vào localStorage** theo định dạng object { id, name } (sử dụng interface để định nghĩa hình thù data)

Id của todo sẽ được random ngẫu nhiên (cần tham số này, để sau này xóa todo)

Ví dụ: data todo khi lưu: { id: 1, name: "learn javascript" }

LocalStorage sẽ lưu dưới định dạng array: `localStorage.setItem(todo, data)`

```
[  
  { id: 1, name: "learn javascript"},  
  { id: 2, name: "learn typescript"},  
  ...  
]
```

- Kết quả todo được lưu dưới dạng string tại localStorage, với hàm [stringify](#)
- Lưu todo thành công, reload lại website (sử dụng javascript)
`window.location.reload();`

Gợi ý function random number (dùng để tạo id của todo)

//random number

```
function getRandomInt(min, max) {  
  min = Math.ceil(min);  
  max = Math.floor(max);  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}
```

#148. Chữa Bài Tập Thực Hành 03 - Part 1

//todo

#149. Bài Tập Thực Hành 03 - Part 2

Tính năng 2: Hiển thị danh sách todo

Hiển thị danh sách todo dưới dạng table, data lấy từ localStorage, tương tự :

Id	Name	Action
1	Learn javascript	Button Xóa
2	Learn typescript	Button Xóa

Gợi ý:

- Sử dụng [table của bootstrap](#) để render nhanh giao diện
- Lấy danh sách todo từ LocalStorage, cần convert từ string sang object, sử dụng [parse](#)
- Sử dụng interface và ép kiểu dữ liệu để tạo hình thù data

#150. Chữa Bài Tập Thực Hành 03 - Part 2

//todo

#151. Bài Tập Thực Hành 03 - Part 3

Tính năng 3: Xử lý hành động xóa todo

Khi nhấn vào button xóa trong table, sẽ tiến hành xóa todo. Sử dụng JavaScript để:

1. Lấy id của todo cần xóa

Gợi ý:

//với html, gán [data attribute](#)

```
<td><button class="delete-btn" data-id="1">Xóa</button></td>
```

// với javascript gán sự kiện click cho tất cả nút có class "delete-btn"

```
document.querySelectorAll('.delete-btn').forEach(function(button) {  
  button.addEventListener('click', function() {  
    const id = button.getAttribute('data-id');  
  });  
});
```

2. Sử dụng hàm **filter** để xóa todo (theo id) - List todo đang lưu tại localStorage
3. Xóa thành công, reload lại website để thấy kết quả

#152. Chữa Bài Tập Thực Hành 03 - Part 3

//todo

#153. Bài Tập Thực Hành 03 - Part 4

Các yêu cầu nâng cao để tăng tính trải nghiệm của người dùng (UX):

1. **Khi xóa todo**, thay vì reload lại website, cần:

- Xóa row trên table (ứng với id của todo cần xóa)

Gợi ý:

```
document.querySelectorAll('.delete-btn').forEach(function(button) {  
  button.addEventListener('click', function() {  
    const row = button.closest('tr');  
    row.remove();  
  });  
});
```

2. **Khi thêm mới todo** thành công, thay vì reload lại website, cần:

- Thêm row mới vào table sử dụng javascript
- Gán sự kiện delete vào row vừa được thêm mới

Gợi ý:

```
const tableBody = document.querySelector('#myTable tbody');
```

```
// Tạo phần tử dòng mới
```

```
const newRow = document.createElement('tr');
```

```
// Gán HTML cho dòng
```

```
newRow.innerHTML = `
```

```
  <td>Người mới ${currentId}</td>
```

```
  <td><button class="delete-btn" data-id="${currentId}">Xóa</button></td>  
`;
```

```
// Thêm dòng vào cuối bảng
```

```
tableBody.appendChild(newRow);
```

3. [Sử dụng toast](#) để hiển thị thông báo khi thao tác thành công:

- Thêm todo thành công
- Xóa todo thành công

#154. Chữa Bài Tập Thực Hành 03 - Part 4

//todo

Chapter 16: Dự Án Thực Hành TypeScript (Project 04)

Thực hành dự án với ngôn ngữ TypeScript (Project 04)

#155. Demo Dự án Thực Hành 04

//todo

#156. Hướng Dẫn Tạo Dự Án Thực Hành 04

1. Tạo base cấu trúc dự án thực hành

Download base dự án thực hành 04 [tại đây](#)

th04/

```
|— blog/
|   |— blog.html
|   |— blog.ts
|— post/
|   |— post.html
|   |— post.ts
|— user/
|   |— user.html
|   |— user.ts
|— home.ts
|— home.html
```

Lưu ý: trước khi test, cần chạy dự án: npm run dev

Mục đích là dịch code typescript sang javascript

Phần navbar:

<https://getbootstrap.com/docs/5.3/components/navbar/>

2. Chạy dự án backend

Cách test api, cũng như file json data được lưu (cách tạo backup data)

#157. Bài Tập Thực Hành 04 - Part 1

Tính năng 1: hiển thị danh sách blogs (gọi API)

Bước 1: Bạn cần chạy backend
`npm run dev`

Kiểm tra backend đã hoạt động chưa:
<http://localhost:8000/>

Kiểm tra API đã hoạt động chưa:
<http://localhost:8000/blogs>

Bước 2: Render tables blogs

Thực hiện gọi API với **fetch** (lấy danh sách blogs)

Id	Title	Author	Content
1	...	...	...
2	...	...	...

Render động dữ liệu của table html dựa vào javascript và data của api

#158. Chữa Bài Tập Thực Hành 05 - Part 1

//todo

Tham khảo source code của tính năng 01 [tại đây](#)

#159. Bài Tập Thực Hành 04 - Part 2

Tính năng 2: tạo mới blog

Bước 1: chạy backend

Kiểm tra API đã hoạt động chưa:

<http://localhost:8000/blogs>

Bước 2: Tạo modal add new blog và xử lý logic

Gợi ý: làm tương tự như modal create todo (tại bài thực hành 03)

Quy trình code, bao gồm:

1. tạo modal (code html)

<https://getbootstrap.com/docs/5.3/components/modal/>

2. trigger modal : Tức là nhấn vào nút create blog, cần mở được modal trên

3. Tạo form để nhập thông tin blog, bao gồm: **title, author, content**

4. Khi nhấn nút submit, cần xử lý:

Lấy thông tin điền trên form

Gọi API để thêm mới blog, tham khảo cú pháp:

<https://stackoverflow.com/a/29823632>

POST <http://localhost:8000/blogs>

body truyền lên object { **title, author, content** } => chính là data lấy từ modal

Gợi ý cách gọi api:

```
const rawResponse = await fetch('http://localhost:8000/blogs', {
  method: 'POST',
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({ title, author, content })
});
```

5. Gọi API thành công, đóng modal, clear data điền trên form

Reload lại website để thấy kết quả

Bước 3: tăng trải nghiệm của người dùng

Tạo xong, append new row, hiển thị thông báo với [toast](#)

Gợi ý thêm mới row:

```
const tableBody = document.querySelector('#myTable tbody');
// Tạo phần tử dòng mới
const newRow = document.createElement('tr');

// Gán HTML cho dòng
newRow.innerHTML = `
  <td>Người mới ${currentId}</td>
  <td><button class="delete-btn" data-id="${currentId}">Xóa</button></td>
`;

// Thêm dòng vào cuối bảng
tableBody.appendChild(newRow);
```

Tạo toast:

```
<div id="createBlogToast"
  class="position-fixed top-0 end-0 m-3 toast align-items-center text-bg-success border-0" role="alert"
  aria-live="assertive" aria-atomic="true">
  <div class="d-flex">
    <div class="toast-body">
      Create a blog success...
    </div>
    <button type="button" class="btn-close btn-close-white me-2 m-auto" data-bs-dismiss="toast"
      aria-label="Close"></button>
  </div>
</div>
```

#160. Chữa Bài Tập Thực Hành 04 - Part 2

//todo

Tham khảo source code của tính năng 02 [tại đây](#)

#161. Bài Tập Thực Hành 04 - Part 3

Tính năng 3: xóa blog

Bước 1: chạy backend

Kiểm tra API đã hoạt động chưa:

<http://localhost:8000/blogs>

Bước 2: Xóa blog

1. Lấy id của blog cần xóa

Gợi ý:

//với html, gán [data attribute](#)

```
<td><button class="delete-btn" data-id="1">Xóa</button></td>
```

```
// với javascript gán sự kiện click cho tất cả nút có class "delete-btn"
document.querySelectorAll('.delete-btn').forEach(function(button) {
  button.addEventListener('click', function() {
    const id = button.getAttribute('data-id');
  });
});
```

2. Gọi api để xóa blog

Xóa 1 blog (tham khảo sử dụng với fetch [tại đây](#))

DELETE <http://localhost:8000/blogs/:id>

Gợi ý: (**cần truyền vào id của blog** cần xóa vào url api)

```
await fetch('http://localhost:8000/blogs/ID', {
  method: 'DELETE',
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json'
  }
});
```

Xóa thành công, reload lại website để thấy kết quả.

Bước 3: tăng trải nghiệm của người dùng

Xóa xong, remove current row, hiển thị thông báo với [toast](#)

Dùng javascript để xóa row:

```
document.querySelectorAll('.delete-btn').forEach(function(button) {  
  button.addEventListener('click', function() {  
    const row = button.closest('tr');  
    row.remove();  
  
  });  
});
```

#162. Chữa Bài Tập Thực Hành 04 - Part 3

//todo

Tham khảo source code của tính năng 03 [tại đây](#)

#163. Bài Tập Thực Hành 04 - Part 4

Tính năng 4: cập nhật blog

Bước 1: chạy backend

Kiểm tra API đã hoạt động chưa:

<http://localhost:8000/blogs>

Bước 2: Tạo modal edit blog và xử lý logic

Gợi ý: làm tương tự như modal create, điểm khác biệt là cần gán giá trị khởi tạo cho input

Quy trình code, bao gồm:

1. tạo modal (code html)

<https://getbootstrap.com/docs/5.3/components/modal/>

2. trigger modal : Tức là nhấn vào nút edit, cần mở được modal trên (có thể dùng javascript để show modal - làm ngược lại so với hide)

3. Tạo form để nhập thông tin blog, bao gồm: **title, author, content**

Cần gán giá trị ban đầu cho các ô input (có thể làm tương tự như cách gán id, đặt thêm các data attributes cho button)

```
<button
  class="btn btn-warning edit-blog"
  data-update-id="${blog.id}"
  data-title="${blog.title}"
  data-author="${blog.author}"
  data-content="${blog.content}"
>
  Edit
</button>
```

4. Khi nhấn nút submit, cần xử lý:

Lấy thông tin điền trên form

Gọi API để thêm update blog, tham khảo cú pháp:

<https://stackoverflow.com/a/29823632>

PUT <http://localhost:8000/blogs/ID> => cần truyền động id vào url
body truyền lên object { **title, author, content** } => chính là data lấy từ modal
Gợi ý cách gọi api:

```
const rawResponse = await fetch('http://localhost:8000/blogs/ID', {  
  method: 'PUT',  
  headers: {  
    'Accept': 'application/json',  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({ title, author, content })  
});
```

5. Gọi API thành công, đóng modal, clear data điền trên form
Reload lại website để thấy kết quả

Bước 3: tăng tính trải nghiệm của người dùng

Cách đơn giản nhất, là xóa row hiện tại, sau đấy thêm lại row mới vào cuối cùng (tạo cảm giác update)

Đồng thời, sử dụng toast để hiển thị thông báo

#164. Chữa Bài Tập Thực Hành 04 - Part 4

//todo

Tham khảo source code của tính năng 04 [tại đây](#)

#165. Bài Tập Thực Hành 04 - Part 5

Tính năng 5: Xây dựng giao diện tương tự cho **users** và **posts**

Yêu cầu 1: CRUD users

Kiểm tra API đã hoạt động chưa:

<http://localhost:8000/users>

1. Hiển thị danh sách user

```
const res = await fetch("http://localhost:8000/users");  
const data = await res.json() as IUser[];
```

2. Create a user

```
const rawResponse = await fetch('http://localhost:8000/users', {  
  method: 'POST',  
  headers: {  
    'Accept': 'application/json',  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({ name, email })  
});
```

```
const res: IUser = await rawResponse.json();
```

3. Delete a user (cần truyền động id)

```
await fetch(`http://localhost:8000/users/${id}`, {  
  method: 'DELETE',  
  headers: {  
    'Accept': 'application/json',  
    'Content-Type': 'application/json'  
  }  
});
```

4.Update a user (cần truyền động id)

```
await fetch(`http://localhost:8000/users/${idElement.value}`, {  
  method: 'PUT',  
  headers: {  
    'Accept': 'application/json',  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({  
    name: nameElement.value,  
    email: emailElement.value,  
  })  
});
```

Yêu cầu 2: CRUD posts

Kiểm tra API đã hoạt động chưa:

<http://localhost:8000/posts>

1.Hiển thị danh sách post

```
const res = await fetch("http://localhost:8000/posts");  
const data = await res.json() as IPost[];
```

2.Create a post

```
const rawResponse = await fetch('http://localhost:8000/posts', {  
  method: 'POST',  
  headers: {  
    'Accept': 'application/json',  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({ title, content })  
});
```

```
const res: IUser = await rawResponse.json();
```

3. Delete a post (cần truyền động id)

```
await fetch(`http://localhost:8000/posts/${id}`, {  
  method: 'DELETE',  
  headers: {  
    'Accept': 'application/json',  
    'Content-Type': 'application/json'  
  }  
});
```

4. Update a post(cần truyền động id)

```
await fetch(`http://localhost:8000/posts/${idElement.value}`, {  
  method: 'PUT',  
  headers: {  
    'Accept': 'application/json',  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({  
    title: titleElement.value,  
    content: contentElement.value,  
  })  
});
```

#166. Chữa Bài Tập Thực Hành 04 - Part 5

//todo

Tham khảo source code của tính năng 05 [tại đây](#)

@hoidanit

What's next với TypeScript

Đây là khóa học về JavaScript/TypeScript, học các kiến thức nền tảng và cốt lõi của ngôn ngữ JavaScript/TypeScript từ số 0 dành cho người mới bắt đầu.

1. Ưu điểm

✓ **Nắm chắc nền tảng** JavaScript/TypeScript

- Hiểu rõ cú pháp của JavaScript, đây là nền tảng vững chắc khi học TypeScript
- Áp dụng tư duy lập trình hướng đối tượng OOP (class, inheritance, polymorphism, abstract, interface)
- Thực hành Generic, Module...
- Đã có kiến thức vững về syntax và logic, giúp tiếp cận frameworks dễ hơn.

2. Nhược điểm (cần cải thiện)

⚠ Học javascript/typescript thiên về lý thuyết, code mẫu, cần thực hành nhiều hơn nữa.

3. Gợi ý các định hướng phát triển:

Định hướng 1: Phát triển **frontend** của website với JavaScript/TS

Bạn có thể học các thư viện, framework như React.js, Angular, Vue... Đây là những công cụ chuyên nghiệp, giúp bạn giảm thiểu thời gian phát triển giao diện website

Tham khảo khóa học React.js của Hỏi Dân IT [tại đây](#)

Định hướng 2: Phát triển **backend** của website với Node.js

Bạn có thể học các framework làm website phổ biến như Express, Nestjs...

Tham khảo khóa học Node.js Pro của Hỏi Dân IT [tại đây](#)

Định hướng 3: Phát triển **mobile app**

Bạn có thể tìm hiểu về React Native

Tham khảo khóa học React Native của Hỏi Dân IT [tại đây](#)

Định hướng 4: Các ứng dụng khác

JavaScript có thể giúp bạn làm desktop app, các ứng dụng IoT...

Tùy vào mục đích sử dụng, bạn cần tìm hiểu các thư viện/framework hỗ trợ lĩnh vực ấy

Đánh Giá (Review/Rating) Khóa Học

1. Lời cảm ơn:

Mình rất hy vọng bạn thấy khóa học này có giá trị, nhưng dù thế nào đi nữa, hãy để lại đánh giá và chia sẻ trải nghiệm của bạn.

Hãy cho mình biết bạn nghĩ gì và viết đánh giá khi bạn có thể.

Mình cũng luôn sẵn sàng trả lời câu hỏi của bạn - hãy gửi tin nhắn trực tiếp cho mình bất cứ lúc nào.

2. Cách đánh giá (review) khóa học, tham khảo chi tiết [tại đây](#)

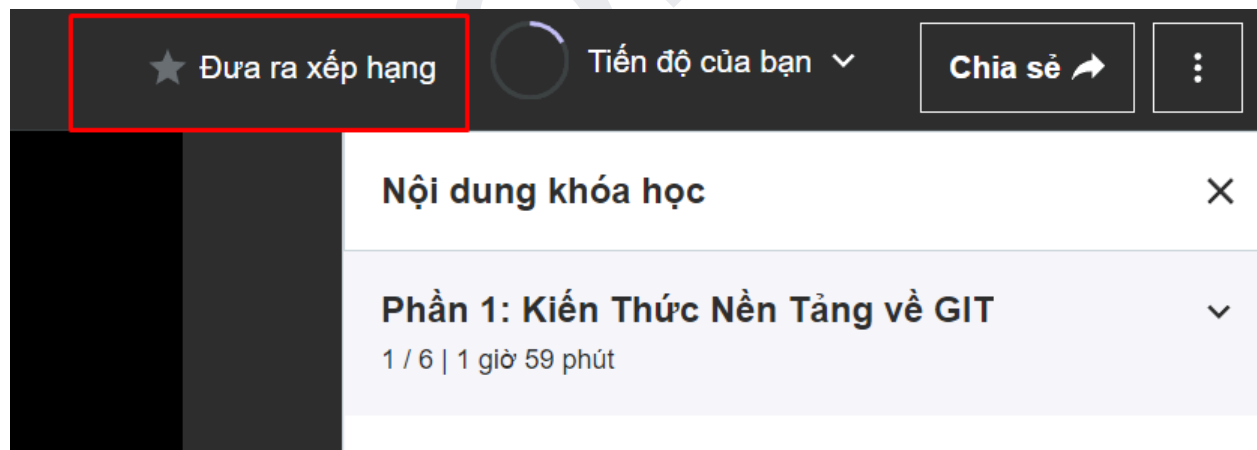
Tạo review đánh giá:

Nhấn vào "đưa ra xếp hạng". Sau đấy rating (star) và viết đánh giá chia sẻ trải nghiệm.

Nếu màn hình của bạn không hiển thị, có 2 nguyên nhân:

1 là bạn đã đánh giá rồi, xem mục 3 bên dưới để biết cách sửa/xóa đánh giá

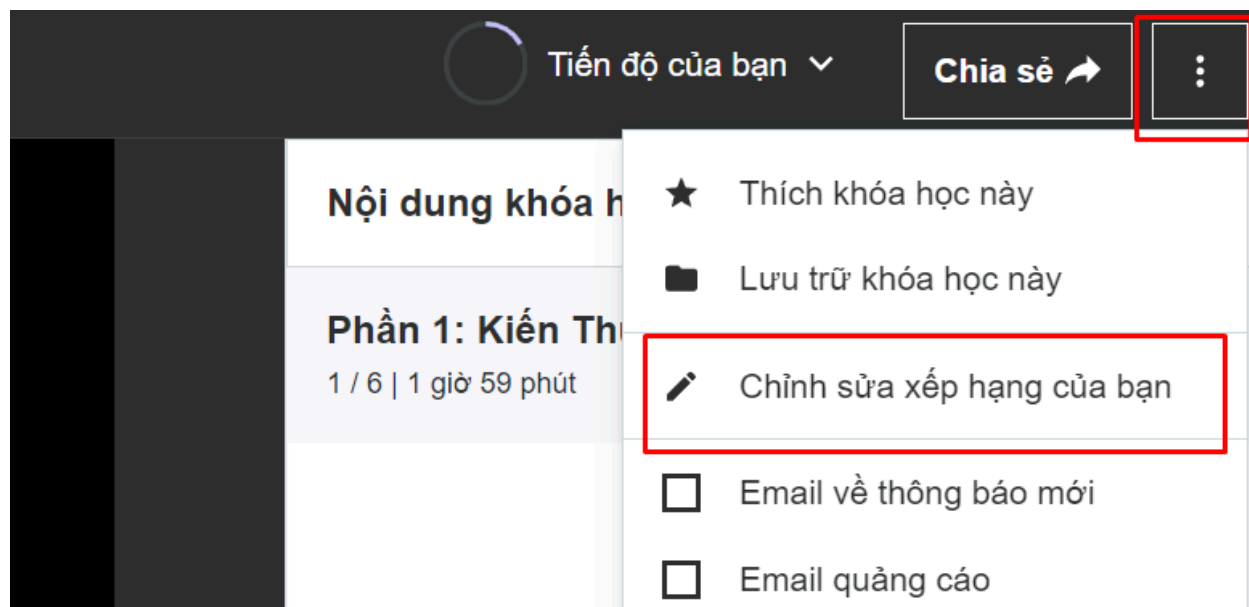
2 là thời gian bạn xem khóa học chưa đủ nên không thể đánh giá



3. Chỉnh sửa/xóa review đã đánh giá:

Bước 1: chọn vào hình 3 dấu chấm tại góc màn hình

Bước 2: chọn chỉnh sửa review



4. Mẹo để Viết Đánh Giá Tốt

Dưới đây là một số điều bạn nên lưu ý khi viết đánh giá cho một khóa học:

Hãy cho chúng tôi biết lý do: Ngoài việc để lại đánh giá sao, vui lòng chia sẻ suy nghĩ của bạn.

Ý kiến của bạn về khóa học rất có giá trị đối với những người học khác, nhưng khi chỉ có đánh giá sao, thật khó để người dùng khác hiểu vì sao bạn lại đưa ra đánh giá đó.

Cụ thể là tốt: Tính cụ thể giúp người học khác quyết định xem khóa học có phù hợp với họ không. Có điểm nào cần cải thiện không?

Khóa học có đáp ứng được kỳ vọng của bạn không?

Phần nào của khóa học khiến bạn thích nhất?

Trung thực: Đánh giá là một trong những yếu tố quan trọng nhất mà mọi người xem xét trước khi đăng ký một khóa học.

Tuy nhiên, đánh giá chỉ có giá trị khi bạn trung thực về cảm nhận của mình về khóa học.

Miễn là bạn chia sẻ cảm nhận một cách tôn trọng, ý kiến phản hồi của bạn rất có giá trị và hữu ích cho cộng đồng học tập của chúng tôi.